FATEC OSASCO PREFEITO HIRANT SANAZAR

Desenvolvimento de Software Multiplataforma

TROCAÍ

Aplicativo de Troca de Habilidades entre Moradores de Condomínios

Entrega 6 — Pitch e conclusão

Objetivo específico 6 · prazo 28/11/2026

Jefferson de Padua Coelho Ribeiro — RA: 2161392413009

Karine Fernandes e Silva — RA: 2161392413030

Sarah Quinteiro Limeira da Silva — RA: 2161392413027

Orientação: Professor Elias Carneiro de Oliveira

Osasco · 2026 / 6° semestre

1. Resumo

Condomínios residenciais reúnem, em poucos metros quadrados, uma diversidade enorme de competências que permanece invisível: moradores contratam serviços externos e pagam por aulas particulares sem saber que o vizinho do andar de cima domina exatamente aquilo. O Trocaí é um aplicativo mobile que torna esse conhecimento visível e trocável. O morador cadastra o que sabe fazer e o que precisa, encontra vizinhos compatíveis dentro do próprio condomínio e combina o encontro nas áreas comuns por um chat interno. As trocas são registradas por créditos virtuais, chamados Elos, sem qualquer envolvimento de dinheiro real: quem ensina acumula, quem aprende gasta. Um painel web complementar entrega ao síndico o ranking de moradores ativos, a gestão de avisos e bloqueios e a Árvore de Talentos, visualização que revela quais habilidades estão em falta na comunidade. O levantamento de requisitos foi conduzido pela metodologia Lean Inception, percorrendo visão de produto, personas, jornada do usuário, brainstorming de funcionalidades, revisão técnica e sequenciamento em ondas. O desenvolvimento prossegue em entregas incrementais, das quais esta documenta a de número 6.

Palavras-chave: Aplicativo Móvel. Mobile Development. Flutter. Economia Colaborativa. Lean Inception. Solução Tecnológica.

2. Introdução

2.1 Tema e justificativa

Viver em condomínio vertical no Brasil significa, hoje, dividir parede com dezenas de pessoas e não conhecer nenhuma delas. Segundo o Instituto Nacional de Condomínios, existem mais de 520 mil condomínios no país, e em cada um deles convivem professores, desenvolvedores, cozinheiros, costureiras, músicos, estudantes e aposentados com décadas de ofício. Esse conjunto de competências é um recurso ocioso: ninguém sabe que ele existe e, por isso, ninguém o utiliza.

O resultado prático é duplo. De um lado, o morador recorre ao mercado externo e paga por um serviço que o vizinho faria de graça; de outro, quem tem tempo e conhecimento disponíveis, como aposentados e estudantes, não encontra canal para oferecê-los nem reconhecimento por fazê-lo. O grupo de WhatsApp, única ferramenta de comunicação da maioria dos prédios, não resolve: pedidos se perdem no volume de mensagens, não há histórico, não há moderação e não existe mecanismo de reciprocidade que garanta que quem ajuda também será ajudado.

A solução precisa ser móvel por três razões. A primeira é de contexto de uso: o pedido nasce no momento em que o problema aparece, longe do computador, e o encontro acontece em uma área comum do próprio prédio. A segunda é de alcance: o smartphone é o único dispositivo que praticamente todos os moradores possuem, o que permite incluir perfis de baixa afinidade digital. A terceira é de notificação: a economia de trocas só funciona se a resposta a um pedido chegar rápido, e apenas o aplicativo entrega isso.

A relevância social está em transformar vizinhança em comunidade sem exigir dinheiro de ninguém, aproveitando um recurso que já existe e hoje se perde. A relevância técnica está no desafio de construir uma economia interna confiável, em que o saldo de créditos nunca divirja, e uma camada de visualização capaz de revelar lacunas de competência a quem administra o condomínio.

2.2 Problema de pesquisa

Como viabilizar, por meio de um aplicativo móvel, a troca de habilidades entre moradores de um mesmo condomínio, de forma confiável e recíproca, sem envolver dinheiro real e sem depender da coincidência entre quem oferece e quem procura?

2.2.1 O produto é, não é, faz, não faz

O produto É	O produto NÃO É
Um ecossistema digital: aplicativo mobile e painel web	Uma rede social aberta ao público
Uma rede fechada e verificada por condomínio	Um marketplace de serviços pagos
Um sistema de economia colaborativa	Uma plataforma de contratação de freelancers
Uma moeda de reciprocidade em créditos virtuais	Uma criptomoeda ou meio de pagamento
Um catálogo vivo dos talentos do prédio	Um sistema de gestão condominial de boletos e atas
Comunitário, local, gratuito e voluntário	Um aplicativo de relacionamento
Simples e acessível, inclusive para idosos	Complexo ou voltado a público técnico
Moderado e transparente para a administração	Anônimo ou obrigatório

O produto FAZ	O produto NÃO FAZ
Cadastra e valida moradores por unidade	Não processa pagamentos nem converte créditos em dinheiro
Divulga as habilidades que cada morador oferece	Não emite nota fiscal nem contrato de prestação de serviço
Registra as necessidades que cada morador procura	Não permite trocas entre condomínios diferentes
Conecta oferta e demanda dentro do mesmo condomínio	Não verifica antecedentes nem certifica profissionalmente
Credita e debita Elos ao concluir a troca	Não garante a qualidade técnica do serviço prestado
Retém os créditos em custódia até a dupla confirmação	Não permite transferir créditos fora de uma troca registrada
Aproxima vizinhos pelo chat interno	Não expõe telefone, e-mail ou número do apartamento
Agenda encontros nas áreas comuns	Não reserva nem controla a ocupação dos espaços
Avalia reputação de forma mútua	Não julga nem arbitra conflitos automaticamente
Publica os avisos do síndico	Não substitui o aplicativo oficial da administradora
Bloqueia trocas em períodos definidos pela administração	Não cancela trocas já aceitas durante o bloqueio
Revela as habilidades em falta pela Árvore de Talentos	Não vende dados dos moradores nem exibe publicidade

2.2.2 Personas

Foram construídas três personas, escolhidas para cobrir os dois lados da economia de trocas e também o público do painel web. O produto só funciona se os três perfis forem atendidos ao mesmo tempo: sem quem oferta não há o que trocar, sem quem solicita os créditos não circulam, e sem o síndico o condomínio não adota a ferramenta.

Camila Duarte, 29 anos, analista de marketing — a Nova no Prédio

"Moro aqui há dois anos e não sei o nome de ninguém do meu andar."

Dimensão	Descrição
Perfil	Mora sozinha no apartamento 74B, trabalha em home office quatro dias por semana. Chegou ao condomínio na pandemia e nunca criou vínculo com os vizinhos. Renda estável, mas orçamento

Dimensão	Descrição
	apertado depois do aumento do aluguel.
Comportamentos	Usa o celular o dia inteiro e resolve quase tudo por aplicativo. Está no grupo de WhatsApp do prédio, mas deixou silenciado. Contrata serviços pelo preço, sem indicação. Evita ligar para desconhecidos. Revisa currículos de amigos de graça, por hábito.
Necessidades	Consertar o notebook sem pagar visita técnica cara. Aprender violão. Sentir-se parte do lugar onde mora. Retribuir o que recebe, para não ficar devendo favor.
Dores	Pediu indicação no grupo e recebeu quinze respostas desencontradas. Receia deixar estranho entrar no apartamento. Acha que não tem nada a oferecer. Desconfia de aplicativo que pede telefone e endereço no cadastro.
Uso do produto	Aplicativo. É o perfil que mais solicita no começo e passa a ofertar quando percebe que a habilidade dela tem demanda.

Antônio Ribeiro, 63 anos, aposentado — o Vizinho Prestativo

"Eu conserto, só não sei mexer nesses aplicativos de hoje."

Dimensão	Descrição
Perfil	Ex-técnico em eletrônica por 34 anos, casado, mora no apartamento 22A há quinze anos. Conhece metade do prédio pelo nome. Aposentadoria apertada.
Comportamentos	Usa o celular para WhatsApp e chamadas com os netos. Aumenta a fonte do sistema ao máximo e ainda tem dificuldade com botões pequenos. Abandona cadastros com mais de três ou quatro campos. Passa a manhã na área comum. Já consertou ventilador, chuveiro e roteador de vizinhos sem cobrar.
Necessidades	Continuar sendo útil e reconhecido. Conseguir ajuda com burocracia digital. Ocupar o tempo livre com propósito. Não depender de favor.
Dores	Sente que ficou invisível depois de se aposentar. Já foi vítima de golpe por link e desconfia de tudo. Cansou de ajudar sempre e nunca receber. Desiste na primeira tela quando a interface tem ícones pequenos.
Uso do produto	Aplicativo. É o ofertante que sustenta a economia de créditos no início; se ele não entra, o sistema não sai do lugar.

Rita Menezes, 45 anos, síndica — a Gestora Sobrecarregada

"Eu decido no achismo porque não tenho dado nenhum sobre o que acontece aqui."

Dimensão	Descrição
Perfil	Advogada, síndica há dois anos, concilia o mandato com trabalho em tempo integral. Administra 96 unidades com salão de festas e espaço gourmet subutilizados.
Comportamentos	Prefere resolver as coisas do condomínio pelo navegador. Apaga incêndio no grupo várias vezes por semana. Prepara a prestação de contas na véspera da assembleia. Publica avisos impressos no elevador.
Necessidades	Reduzir atrito entre moradores. Aumentar o uso das áreas comuns. Ter números concretos para a assembleia. Suspender atividades em períodos críticos. Descobrir o que a comunidade precisa.
Dores	Não sabe quais habilidades existem no prédio. Teme ser responsabilizada se uma troca der errado. Já tentou organizar mutirões e ninguém apareceu. Não tem tempo para moderar mais um canal.
Uso do produto	Painel web. É quem aprova cadastros, publica avisos, bloqueia períodos e consulta a Árvore de Talentos.

2.2.3 Jornada do usuário

As jornadas descrevem o percurso de cada persona em etapas cronológicas, com a emoção de cada momento, as dores críticas e o ponto exato em que o produto entra em cena. A coluna de marcação identifica onde está a dor e onde o valor se realiza.

Jornada 1 — Camila: conserta o notebook

#	Etapa	Marcação
1	acorda e começa a trabalhar em casa	neutra
2	o notebook trava no meio da manhã	neutra
3	perde a manhã tentando resolver sozinha	DOR CRÍTICA
4	pede indicação no grupo do prédio	neutra
5	recebe quinze respostas desencontradas	DOR CRÍTICA
6	abre o Trocaí	entrada do produto
7	busca conserto de computador no prédio	neutra
8	encontra o Seu Antônio, do 22A	alívio
9	combina o encontro pelo chat	alívio
10	paga em créditos e ganha um vizinho	MOMENTO DE VALOR

Contexto de uso: O aplicativo é acionado em casa, no meio do expediente, logo após a tentativa frustrada pelo canal informal do condomínio.

Jornada 2 — Seu Antônio: ser encontrado pelos vizinhos

#	Etapa	Marcação
1	acorda cedo e não tem compromisso	neutra
2	passa a manhã na área comum	neutra
3	um vizinho pede ajuda no corredor	neutra
4	conserta o ventilador e não cobra nada	alívio
5	ninguém registra o que ele fez	DOR CRÍTICA
6	precisa de ajuda e não sabe a quem pedir	DOR CRÍTICA
7	o filho instala o Trocaí para ele	entrada do produto
8	cadastra conserto de eletrônicos	neutra
9	recebe pedidos de vizinhos que não conhecia	alívio
10	usa os créditos para pedir ajuda com o celular	MOMENTO DE VALOR

Contexto de uso: O aplicativo entra como registro do que antes era favor informal, convertendo reconhecimento em saldo utilizável.

Jornada 3 — Rita: saber o que falta no condomínio

#	Etapa	Marcação
1	chega em casa depois do trabalho	neutra
2	abre o grupo do prédio com 200 mensagens	DOR CRÍTICA
3	apaga mais um incêndio entre vizinhos	DOR CRÍTICA
4	passa pelo salão de festas vazio	neutra
5	monta a prestação de contas sem dado nenhum	DOR CRÍTICA
6	acessa o painel do Trocaí	entrada do produto
7	confere o ranking de moradores ativos	neutra
8	a Árvore de Talentos mostra o que falta	alívio
9	publica um aviso e bloqueia a semana da reforma	alívio
10	leva números concretos para a assembleia	MOMENTO DE VALOR

Contexto de uso: Diferentemente das anteriores, esta jornada percorre o painel web e não gera troca alguma; ela alimenta decisão administrativa.

Observação transversal: nas três jornadas as dores críticas se concentram antes da entrada do produto e as emoções positivas só aparecem depois dela. Uma segunda observação, menos confortável, é que nenhuma das jornadas acontece com frequência alta, o que levou à inclusão de oficinas abertas no escopo das ondas seguintes.

2.3 Objetivos

2.3.1 Objetivo geral

Desenvolver um aplicativo mobile, acompanhado de um painel web administrativo, que viabilize a troca de habilidades entre moradores de um mesmo condomínio por meio de um sistema de créditos virtuais sem valor monetário, promovendo economia colaborativa, aproveitamento de competências ociosas e fortalecimento dos laços comunitários.

2.3.2 Objetivos de Desenvolvimento Sustentável e benefícios à sociedade

ODS	Alinhamento
ODS 11 — Cidades e Comunidades Sustentáveis	Alinhamento principal. Fortalece laços de vizinhança, aumenta o uso de espaços comuns ociosos e estimula a resolução local de necessidades.
ODS 12 — Consumo e Produção Responsáveis	Substitui a contratação externa pelo aproveitamento de competências já existentes no prédio, reduzindo consumo desnecessário.
ODS 4 — Educação de Qualidade	Boa parte das trocas é de ensino entre pares, ampliando o acesso a conhecimento sem barreira financeira.
ODS 10 — Redução das Desigualdades	A moeda do sistema é o tempo, não o dinheiro, o que dá poder de troca igual a quem tem renda alta e a quem tem renda baixa.

Benefícios à sociedade:

- Para o morador: economia direta em serviços cotidianos, acesso a aprendizado gratuito e redução do isolamento social

- Para o condomínio: ocupação das áreas comuns, redução de conflitos e informação concreta para a assembleia
- Para a comunidade ampliada: valorização de saberes não formais e um modelo replicável de economia solidária

2.3.3 Objetivos específicos — cronograma

As seis entregas abaixo organizam o desenvolvimento até o fim do semestre. A coluna de situação indica o andamento no momento desta entrega.

Data	Objetivo específico	Situação
17/10/2026	Formalizar os requisitos funcionais e não funcionais em histórias de usuário com critérios de aceitação e elaborar o diagrama de casos de uso dos dois perfis	Entregue
24/10/2026	Desenvolver a interface no Figma, com protótipo navegável das telas principais, atendendo aos requisitos de acessibilidade levantados na persona de baixa afinidade digital	Entregue
31/10/2026	Definir a base de dados, entregando o modelo entidade-relacionamento, o dicionário de dados e as regras de integridade da carteira de créditos	Entregue
07/11/2026	Implementar o núcleo do aplicativo em Flutter, integrado a uma API em Node.js com PostgreSQL, contemplando as funcionalidades da Onda 1	Entregue
14/11/2026	Realizar testes funcionais documentados e validação com usuários em um condomínio piloto, registrando defeitos, feedback e taxa de conclusão das trocas	Entregue
28/11/2026	Criar o pitch de apresentação e concluir a documentação do projeto, consolidando resultados, limitações e propostas de evolução	Esta entrega

3. Fundamentação Teórica

3.1 Desenvolvimento nativo, híbrido e multiplataforma

O desenvolvimento nativo consiste em construir o aplicativo na linguagem e no SDK próprios de cada plataforma, Kotlin e Jetpack Compose no Android e Swift e SwiftUI no iOS. Entrega o melhor desempenho e acesso imediato a recursos novos do sistema, mas exige duas bases de código distintas e, consequentemente, duas equipes ou o dobro do tempo. O desenvolvimento híbrido tradicional, baseado em WebView, resolve o custo mas penaliza a fluidez da interface. A abordagem multiplataforma compilada, adotada por Flutter e React Native, ocupa o meio-termo: uma única base de código gera artefatos para Android e iOS com desempenho próximo ao nativo. Para um projeto acadêmico com equipe reduzida e prazo de um semestre, essa é a escolha tecnicamente justificada.

3.2 Flutter e Dart

O Flutter é um framework de código aberto mantido pelo Google que compila Dart para código nativo ARM e desenha a própria interface por meio do motor gráfico Skia, em vez de delegar aos componentes do sistema. Essa decisão arquitetural garante que a aparência seja idêntica nas duas plataformas e dá controle fino sobre acessibilidade, ponto relevante para este projeto por causa da persona de baixa afinidade digital. O modelo de widgets declarativo e o recurso de recarga a quente reduzem o ciclo de iteração durante o desenvolvimento.

3.3 Interface e experiência do usuário em aplicações móveis

A literatura de usabilidade móvel converge em alguns princípios que orientam este projeto. As diretrizes do Material Design recomendam área mínima de toque de 48 por 48 pixels independentes de densidade, o que afeta diretamente a usabilidade para usuários idosos. As WCAG 2.1, no nível AA, estabelecem razão de contraste mínima de 4,5 para 1 em texto comum e exigem que o conteúdo permaneça legível com a fonte ampliada em até 200 por cento. A heurística de Nielsen sobre visibilidade do estado do sistema justifica a exibição permanente do saldo de créditos, e a de reconhecimento em vez de memorização sustenta a navegação por categorias além da busca textual pura.

3.4 Persistência de dados em aplicações móveis

Aplicações móveis costumam combinar armazenamento local e remoto. O SQLite, embarcado no dispositivo, atende cache e funcionamento offline. No lado remoto há duas famílias. Soluções NoSQL gerenciadas, como o Firebase Firestore, oferecem sincronização em tempo real e baixo custo inicial de configuração, mas não garantem transações multi-documento com a mesma robustez de um banco relacional. Bancos relacionais, como o PostgreSQL, oferecem transações ACID, restrições de integridade e consistência forte. Como a carteira de créditos do Trocaí não pode divergir em nenhuma hipótese, uma transferência de Elos precisa ser atômica: ou debita e credita, ou não faz nada. Esse requisito pesa a favor do modelo relacional para o núcleo da aplicação.

3.5 Economia colaborativa e bancos de tempo

O conceito de banco de tempo, formalizado por Edgar Cahn na década de 1980, propõe uma moeda complementar em que a unidade de valor é a hora de serviço prestado, igual para qualquer tipo de trabalho. A premissa é que a reciprocidade registrada reduz o constrangimento de pedir ajuda e torna sustentável o que, informalmente, se esgota. A literatura sobre essas iniciativas também registra o principal fator de fracasso: a falta de liquidez. Em comunidades pequenas, oferta e demanda raramente coincidem no tempo, e participantes acumulam créditos sem conseguir gastá-los. Esse achado foi incorporado ao projeto e motivou a proposta de oficinas abertas, em que uma única oferta atende vários moradores simultaneamente.

3.6 Lean Inception

A Lean Inception, sistematizada por Paulo Caroli, é um workshop colaborativo que combina Design Thinking e Lean Startup para definir o produto mínimo viável. Estrutura-se em atividades sequenciais que partem da visão de produto e avançam por É-Não É-Faz-Não Faz, objetivos, personas, jornadas, brainstorming de funcionalidades, revisão técnica e de negócio, sequenciador e canvas do MVP. A metodologia foi adotada como instrumento de levantamento de requisitos porque garante rastreabilidade: cada funcionalidade proposta tem origem em uma oportunidade identificada em uma jornada, que por sua vez pertence a uma persona definida.

4. Metodologia

4.1 Tipo de pesquisa

Trata-se de pesquisa aplicada, de natureza exploratória e abordagem qualitativa, materializada no desenvolvimento de um aplicativo mobile como estudo de caso.

A fase de levantamento de requisitos foi conduzida pela metodologia Lean Inception, em sessões remotas facilitadas com uso de quadro digital colaborativo. Cada atividade seguiu o mesmo protocolo:

contextualização pelo facilitador, geração silenciosa de ideias com tempo cronometrado para evitar que apenas os participantes mais comunicativos definissem o resultado, agrupamento por afinidade e consolidação por consenso. Nas decisões de divergência, adotou-se o critério de que questões técnicas são decididas pelos desenvolvedores e questões de valor pelo responsável pelo produto.

A fase de construção adota desenvolvimento incremental organizado em seis entregas quinzenais, conforme o cronograma da seção 2.3.3, com validação ao final de cada uma.

4.2 Ferramentas e tecnologias

Camada	Tecnologia	Justificativa
Aplicativo mobile	Flutter com Dart	Base de código única para Android e iOS, com controle fino sobre acessibilidade
Painel web	React com TypeScript	Ecossistema maduro e bibliotecas de visualização de dados
Backend	Node.js com Express, API REST	Produtividade e mesma linguagem do front web
Banco de dados	PostgreSQL	Transações ACID, indispensáveis para a carteira de créditos
Testes	Jest	Execução rápida e relatório de cobertura integrado
Notificações	Firebase Cloud Messaging	Push gratuito
Design	Figma e FigJam	Protótipos, design system e quadro colaborativo
Versionamento	Git e GitHub Actions	Fluxo de revisão por pares e integração contínua

4.3 Levantamento de requisitos

4.3.1 Requisitos funcionais

Código	Descrição
RF01	Cadastrar conta informando nome, unidade e código do condomínio
RF02	Validar o morador por aprovação do síndico antes de liberar o acesso
RF03	Cadastrar habilidades oferecidas, com categoria, descrição e custo em créditos
RF04	Cadastrar necessidades procuradas
RF05	Buscar habilidades por palavra-chave e filtrar por categoria, bloco e disponibilidade
RF06	Enviar, aceitar, recusar e cancelar solicitações de troca
RF07	Reter os créditos em custódia entre o aceite e a conclusão da troca
RF08	Exigir confirmação das duas partes para dar a troca por concluída
RF09	Transferir os créditos automaticamente após a dupla confirmação
RF10	Exibir o saldo na tela inicial e o extrato de entradas e saídas
RF11	Bloquear novas solicitações quando o saldo estiver abaixo do limite de dívida
RF12	Trocar mensagens por chat interno vinculado a cada troca
RF13	Agendar data, horário e área comum do encontro
RF14	Avaliar o vizinho com nota e comentário após a conclusão
RF15	Denunciar conduta inadequada e permitir suspensão do usuário
RF16	Notificar novas solicitações, mensagens e avisos

Código	Descrição
RF17	Aprovar ou recusar cadastros de moradores pelo painel web
RF18	Publicar e agendar avisos para todo o condomínio
RF19	Bloquear novas trocas em um intervalo de datas, preservando as já aceitas
RF20	Exibir o ranking de moradores mais ativos e a Árvore de Talentos com as habilidades em falta
RF21	Exibir o perfil público do vizinho, com habilidades, avaliação média e trocas concluídas
RF22	Listar as solicitações recebidas em ordem de chegada
RF23	Marcar períodos de indisponibilidade no próprio perfil
RF24	Ordenar os resultados da busca por avaliação e destacar novos ofertantes
RF25	Informar a duração estimada da troca no momento da solicitação
RF26	Conceder selos por marcos de participação

4.3.2 Requisitos não funcionais

Código	Descrição
RNF01	Compatibilidade: Android 8.0 ou superior e iOS 13 ou superior; painel web responsivo
RNF02	Desempenho: resposta da API abaixo de 500 ms em 95% das requisições; mensagem no chat abaixo de 1 segundo
RNF03	Confiabilidade: movimentação de créditos em transação atômica, com registro imutável, sem divergência de saldo
RNF04	Segurança: senhas com hash, autenticação por token com expiração e acesso restrito a moradores validados
RNF05	Privacidade e LGPD: consentimento explícito, exibição pública limitada a nome e bloco, contato só pelo chat interno, direito à exclusão com anonimização
RNF06	Acessibilidade: WCAG 2.1 nível AA, leitor de tela, fonte ampliada e área de toque mínima de 48 por 48 dp
RNF07	Usabilidade: cadastro completo em até três minutos; fluxo principal em no máximo quatro toques
RNF08	Disponibilidade: 99% no intervalo das 6h às 23h
RNF09	Restrição de domínio: nenhuma transação financeira real; créditos não podem ser comprados, vendidos ou convertidos
RNF10	Escalabilidade: modelo de dados multi-tenant, preparado para vários condomínios de forma isolada
RNF11	Manutenibilidade: cobertura mínima de 60% de testes automatizados no backend e padronização por linter

4.3.3 Regras de negócio da economia de Elos

Código	Descrição
RN01	1 Elo equivale a 30 minutos de dedicação. É referência sugerida, negociável entre as partes
RN02	Todo novo morador aprovado recebe 3 Elos de boas-vindas
RN03	Elos não têm valor monetário e não podem ser comprados, vendidos nem convertidos em dinheiro
RN04	O débito e o crédito só ocorrem após a confirmação de ambas as partes
RN05	O saldo pode ficar negativo até o limite de -3 Elos; abaixo disso o morador só pode ofertar
RN06	Elos ficam em custódia desde o aceite da troca até a conclusão ou o cancelamento
RN07	Cancelamento antes do encontro devolve integralmente os Elos retidos
RN08	Divergência na confirmação abre um chamado para mediação do síndico

Código	Descrição
RN09	Elos são pessoais e intransferíveis fora de uma troca registrada
RN10	Durante um período bloqueado, novas solicitações ficam indisponíveis; trocas aceitas permanecem válidas
RN11	Teto de 10 trocas concluídas por morador por mês, para evitar profissionalização disfarçada

5. Desenvolvimento (resultados e discussões)

Esta entrega final consolida, em um único documento, o resultado dos cinco objetivos específicos anteriores, revisados à luz do que se aprendeu durante o desenvolvimento. As subseções 5.1 a 5.5 reproduzem cada uma das entregas parciais.

5.1 Histórias de Usuário e Casos de Uso

As 36 funcionalidades classificadas na revisão técnica, de negócio e de UX da Lean Inception foram traduzidas em histórias de usuário no formato "Como [papel], quero [ação], para [benefício]", cada uma acompanhada de critérios de aceitação escritos na notação Dado / Quando / Então. A escolha pelo formato de histórias em vez de uma lista de requisitos isolados preserva o motivo de cada funcionalidade existir, o que é determinante num produto cuja adoção depende de comportamento e não de obrigação.

As histórias foram agrupadas em épicos e priorizadas conforme o sequenciador de ondas definido na fase anterior. A coluna Onda indica em qual entrega a história está prevista.

5.1.1 Épico 1 — Acesso e identidade

ID	História	Critérios de aceitação	RF	Onda
US01	Como morador, quero me cadastrar informando minha unidade e o código do condomínio, para que apenas quem mora aqui participe.	Dado que informei nome, unidade e código válido, quando envio o cadastro, então a conta fica pendente de aprovação e vejo um aviso explicando isso. · Dado que o código está incorreto, quando envio, então recebo mensagem de erro e permaneço na tela com os dados preenchidos. · O formulário não pode exigir mais de três campos obrigatórios.	RF01	1
US02	Como síndico, quero aprovar ou recusar cadastros, para garantir que apenas moradores reais tenham acesso.	Dado que existe cadastro pendente, quando acesso o painel, então vejo nome, unidade e data da solicitação. · Quando aprovo, então o morador recebe aviso e 3 Elos de boas-vindas. · Quando recuso, então registro o motivo e a conta é bloqueada.	RF02, RF17	1
US03	Como morador, quero ver o perfil público de um vizinho, para decidir se confio nele antes de pedir uma troca.	Dado que abro um perfil, então vejo nome, bloco, habilidades, avaliação média e número de trocas concluídas. · O perfil não exibe telefone, e-mail nem número exato do apartamento.	RF21	1
US04	Como morador com pouca familiaridade digital, quero uma interface com fonte ampliada e botões grandes, para conseguir usar o aplicativo sem ajuda.	Dado que ativo o modo de fonte ampliada, então todos os textos aumentam sem cortar conteúdo. · Nenhuma área de toque é menor que 48 por 48 dp. · O contraste atende WCAG 2.1 nível AA.	RNF06	1

5.1.2 Épico 2 — Catálogo de talentos

ID	História	Critérios de aceitação	RF	Onda
US05	Como morador, quero cadastrar o que sei fazer, para que meus vizinhos saibam que podem me procurar.	Dado que preencho título, categoria e descrição, quando salvo, então a habilidade aparece na busca imediatamente. · Posso definir o custo em Elos, com valor sugerido pelo sistema conforme a duração informada. · Posso desativar uma habilidade sem apagá-la.	RF03	1
US06	Como morador, quero registrar o	Dado que registro uma necessidade, então ela alimenta	RF04	1

ID	História	Critérios de aceitação	RF	Onda
	que estou procurando, para que o sistema saiba o que falta no condomínio.	o relatório de lacunas do síndico. · Se já existe oferta correspondente, então recebo sugestão de vizinhos na mesma tela.		
US07	Como morador, quero marcar períodos em que não posso atender, para não receber pedidos quando estiver indisponível.	Dado que marco um período, então minhas habilidades ficam sinalizadas como indisponíveis na busca. · Pedidos já aceitos não são afetados.	RF23	3

5.1.3 Épico 3 — Descoberta

ID	História	Critérios de aceitação	RF	Onda
US08	Como morador, quero buscar uma habilidade por palavra-chave, para encontrar rapidamente quem resolve meu problema.	Dado que digito um termo, então vejo apenas moradores do meu condomínio que oferecem algo correspondente. · A busca ignora acentuação e diferenças de maiúscula. · Se não houver resultado, então vejo a opção de registrar a necessidade.	RF05	1
US09	Como morador, quero filtrar os resultados por categoria e bloco, para escolher alguém próximo e do tipo certo.	Dado que aplico filtros, então a lista é atualizada sem recarregar a tela. · Os filtros aplicados ficam visíveis e podem ser limpos com um toque.	RF05	2
US10	Como morador, quero ver primeiro quem é melhor avaliado, para reduzir o risco da primeira troca.	Dado que escolho ordenar por avaliação, então os resultados são reordenados da maior para a menor nota. · Moradores sem avaliação aparecem com o rótulo "ainda sem avaliações" e não no fim da lista.	RF24	3

5.1.4 Épico 4 — Troca

ID	História	Critérios de aceitação	RF	Onda
US11	Como morador, quero solicitar uma troca a um vizinho específico, para pedir ajuda sem expor meu problema a todo o condomínio.	Dado que escolho uma habilidade, quando envio a solicitação, então descrevo a necessidade e vejo o custo em Elos antes de confirmar. · O ofertante é notificado. · A solicitação fica com status pendente.	RF06, RF25	1
US12	Como morador ofertante, quero ver os pedidos recebidos em ordem de chegada, para responder sem perder nenhum.	Dado que tenho pedidos pendentes, então vejo a lista ordenada do mais antigo ao mais recente, com indicador de não lido.	RF22	2
US13	Como morador ofertante, quero aceitar ou recusar um pedido, para controlar minha agenda.	Dado que aceito, então os Elos do solicitante são reservados e o chat é liberado. · Dado que recuso, então o solicitante é avisado e nenhum crédito é movimentado.	RF06	1
US14	Como participante de uma troca, quero confirmar que ela aconteceu, para que os créditos sejam transferidos com justiça.	Dado que as duas partes confirmam, então os créditos são transferidos e a troca passa a concluída. · Dado que apenas uma confirma, então a troca fica em pendência de confirmação e nada é movimentado. · Dado que há divergência após sete dias, então um chamado de mediação é aberto para o síndico.	RF08, RF09	1
US15	Como morador, quero cancelar uma troca justificando o motivo, para não travar o saldo de ninguém.	Dado que cancelo antes do encontro, então os Elos retidos são devolvidos integralmente e a outra parte é avisada com a justificativa.	RF06	2

5.1.5 Épico 5 — Carteira de Elos

ID	História	Critérios de aceitação	RF	Onda
US16	Como morador, quero ver meu saldo na tela inicial, para saber se posso pedir alguma coisa.	Dado que abro o aplicativo, então vejo o saldo atual e quantos Elos estão retidos em trocas em andamento.	RF10	1
US17	Como morador, quero consultar o extrato das minhas movimentações, para entender para onde foram meus créditos.	Dado que abro o extrato, então vejo data, tipo, valor, saldo após a operação e a troca que originou cada lançamento. · O extrato é somente leitura.	RF10	2
US18	Como sistema, quero impedir pedidos de quem está abaixo do limite de dívida, para manter a economia equilibrada.	Dado que o saldo é menor que -3 Elos, quando tento solicitar, então a ação é bloqueada com explicação de que preciso ofertar para voltar a pedir.	RF11	2
US19	Como sistema, quero reter os créditos entre o aceite e a conclusão, para que ninguém gaste o que já prometeu.	Dado que uma troca é aceita, então os Elos saem do saldo disponível e entram em custódia. · Na conclusão, seguem para o ofertante; no cancelamento, voltam ao solicitante.	RF07	futura

5.1.6 Épico 6 — Comunicação e reputação

ID	História	Critérios de aceitação	RF	Onda
US20	Como participante de uma troca, quero conversar com o vizinho dentro do aplicativo, para combinar o encontro sem passar meu telefone.	Dado que a troca foi aceita, então o chat fica disponível e vinculado àquela troca. · O histórico permanece acessível após a conclusão. · O chat é encerrado se a troca for cancelada.	RF12	1
US21	Como participante, quero registrar data, hora e área comum do encontro, para que ambos tenham a mesma informação.	Dado que defino o agendamento, então as duas partes veem os mesmos dados no topo da conversa.	RF13	1
US22	Como morador, quero avaliar o vizinho após a troca, para ajudar os próximos a decidirem.	Dado que a troca foi concluída, então posso dar nota de 1 a 5 e comentário opcional. · A avaliação só é liberada após a dupla confirmação. · A média aparece no perfil do avaliado.	RF14	1
US23	Como morador, quero denunciar conduta inadequada, para me sentir seguro usando o aplicativo.	Dado que denuncio, então o síndico recebe o caso com o histórico da troca. · Posso bloquear o usuário imediatamente, sem esperar a moderação.	RF15	2

5.1.7 Épico 7 — Gestão do condomínio

ID	História	Critérios de aceitação	RF	Onda
US24	Como síndico, quero publicar avisos para todos os moradores, para substituir o cartaz do elevador.	Dado que publico um aviso, então ele aparece na tela inicial de todos. · Posso agendar início e fim de exibição.	RF18	2
US25	Como síndico, quero bloquear novas trocas em um intervalo de datas, para suspender a atividade durante a reforma.	Dado que defino um período, então novas solicitações ficam indisponíveis nesse intervalo. · Trocas já aceitas permanecem válidas e podem ser concluídas.	RF19	2
US26	Como síndico, quero ver quais habilidades faltam no condomínio,	Dado que acesso o painel, então vejo as habilidades com demanda registrada e nenhuma oferta, ordenadas	RF20	2

ID	História	Critérios de aceitação	RF	Onda
	para decidir onde investir uma oficina.	por número de pedidos. · Posso disparar um aviso segmentado a partir dessa lista.		
US27	Como síndico, quero o ranking dos moradores mais ativos, para reconhecer quem sustenta a comunidade.	Dado que acesso o ranking, então vejo trocas concluídas e avaliação média por morador no período escolhido.	RF20	2

5.1.8 Diagrama de casos de uso

O diagrama a seguir representa os dois atores do sistema e os casos de uso com os quais interagem. O morador concentra as ações de catálogo e troca; o síndico, as de governança. Não há herança entre os atores: o síndico é também morador, mas acessa o painel por um perfil distinto, o que foi modelado como associação separada para deixar explícito que as funções de gestão não se misturam às de troca.

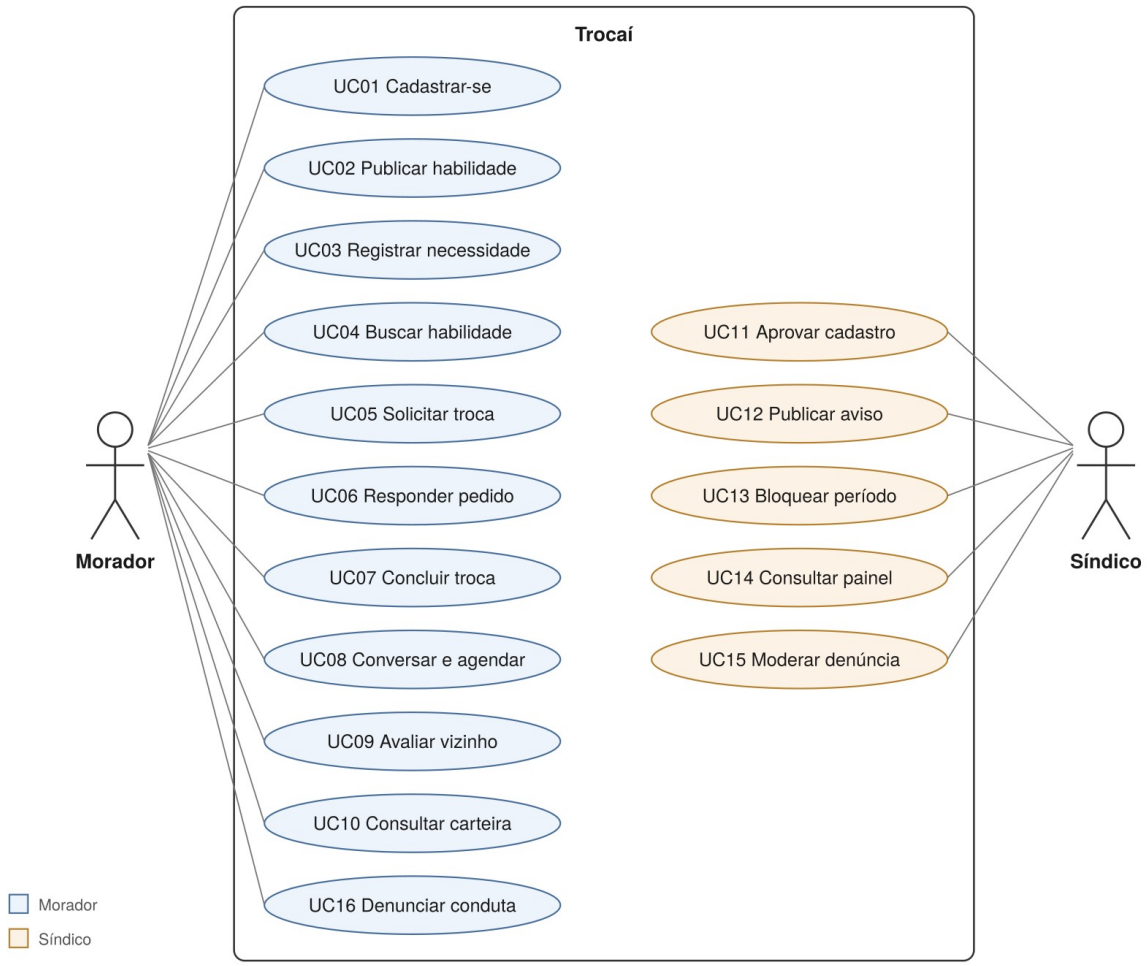


Figura 1 — Diagrama de casos de uso do Trocaí

5.1.9 Especificação dos casos de uso críticos

Os quatro casos de uso abaixo foram detalhados por concentrarem as regras de negócio do produto. Os demais seguem fluxo direto de cadastro e consulta.

UC01 — Cadastrar-se no condomínio

Campo	Descrição
Ator principal	Morador
Pré-condição	O condomínio está cadastrado e o morador possui o código de acesso divulgado pelo síndico
Pós-condição	Conta criada com status pendente, aguardando aprovação
Fluxo principal	1. O morador informa nome, unidade e código do condomínio. 2. O sistema valida o código. 3. O sistema cria a conta com status pendente. 4. O sistema notifica o síndico. 5. O sistema exibe tela explicando a espera pela aprovação.
Fluxo alternativo A1	Código inválido: o sistema informa o erro e mantém os dados preenchidos.
Fluxo alternativo A2	Unidade já cadastrada por outro morador: o sistema permite prosseguir, pois uma unidade pode ter mais de um morador, e sinaliza o caso ao síndico.
Regras	RN02 — na aprovação, o morador recebe 3 Elos de boas-vindas. RNF07 — no máximo três campos obrigatórios.

UC05 — Solicitar troca

Campo	Descrição
Ator principal	Morador solicitante
Pré-condição	Morador autenticado e aprovado, saldo acima do limite de dívida, habilidade ativa e fora de período bloqueado
Pós-condição	Solicitação registrada com status pendente e ofertante notificado
Fluxo principal	1. O morador seleciona uma habilidade. 2. O sistema exibe o custo em Elos e o saldo atual. 3. O morador descreve a necessidade e informa a duração estimada. 4. O sistema registra a solicitação. 5. O sistema notifica o ofertante.
Fluxo alternativo A1	Saldo abaixo de -3 Elos: o sistema bloqueia a ação e explica que é preciso ofertar para voltar a pedir (RN05).
Fluxo alternativo A2	Período bloqueado pelo síndico: o sistema informa a suspensão e a data de retomada (RN10).
Fluxo alternativo A3	Teto mensal atingido: o sistema informa o limite de 10 trocas por mês (RN11).
Regras	RN01, RN05, RN10, RN11

UC07 — Concluir troca e transferir créditos

Campo	Descrição
Ator principal	Morador solicitante e morador ofertante
Pré-condição	Troca com status aceito e encontro realizado
Pós-condição	Troca concluída, créditos transferidos e avaliação liberada
Fluxo principal	1. Uma das partes confirma a conclusão. 2. O sistema registra a confirmação e aguarda a outra parte. 3. A segunda parte confirma. 4. O sistema executa a transferência em transação atômica: debita o solicitante, credita o ofertante e grava duas linhas em movimentacao_elos. 5. O sistema altera o status para concluída. 6. O sistema libera a avaliação mútua.
Fluxo alternativo A1	Apenas uma parte confirma: a troca permanece em pendência de confirmação e nenhum crédito é movimentado.
Fluxo alternativo A2	Divergência após sete dias: o sistema abre chamado de mediação para o síndico (RN08).
Fluxo de exceção	Falha durante a transferência: a transação é revertida integralmente e nenhum saldo é alterado

Campo	Descrição
E1	(RNF03).
Regras	RN04, RN08. Este é o caso de uso de maior risco técnico do sistema: a transferência precisa ser atômica, sob pena de divergência permanente de saldo.

UC13 — Bloquear período de trocas

Campo	Descrição
Ator principal	Síndico
Pré-condição	Síndico autenticado no painel web
Pós-condição	Período registrado; novas solicitações indisponíveis no intervalo
Fluxo principal	1. O síndico informa data inicial, data final e motivo. 2. O sistema valida que a data final é posterior à inicial. 3. O sistema registra o bloqueio. 4. O sistema publica aviso automático aos moradores.
Fluxo alternativo A1	Já existe bloqueio sobreposto: o sistema sugere estender o bloqueio existente em vez de criar outro.
Regras	RN10 — trocas já aceitas permanecem válidas e podem ser concluídas durante o bloqueio.

5.1.10 Matriz de rastreabilidade

A matriz liga cada requisito funcional à história que o detalha e ao caso de uso que o executa, permitindo verificar, em qualquer ponto do desenvolvimento, se algum requisito ficou sem cobertura.

RF	História	Caso de uso	Onda
RF01	US01	UC01	1
RF02	US02	UC01, UC11	1
RF03	US05	UC02	1
RF04	US06	UC03	1
RF05	US08, US09	UC04	1 e 2
RF06	US11, US13, US15	UC05, UC06	1 e 2
RF07	US19	UC07	futura
RF08	US14	UC07	1
RF09	US14	UC07	1
RF10	US16, US17	UC10	1 e 2
RF11	US18	UC05	2
RF12	US20	UC08	1
RF13	US21	UC08	1
RF14	US22	UC09	1
RF15	US23	UC16, UC15	2
RF16	US12	UC06	2
RF17	US02	UC11	2
RF18	US24	UC12	2
RF19	US25	UC13	2
RF20	US26, US27	UC14	2

RF	História	Caso de uso	Onda
RF21	US03	UC04	1
RF22	US12	UC06	2
RF23	US07	UC02	3
RF24	US10	UC04	3
RF25	US11	UC05	3
RF26	—	UC10	3

Os 26 requisitos funcionais estão cobertos por 27 histórias de usuário e 16 casos de uso. O RF26 não gerou história própria por ser um comportamento automático do sistema, sem interação direta do usuário.

5.2 Protótipo e UX/UI

O desenho da interface partiu de uma restrição que atravessa todo o projeto: o produto precisa funcionar ao mesmo tempo para Camila, que resolve tudo por aplicativo, e para Seu Antônio, que abandona qualquer cadastro com mais de três ou quatro campos. Como o Seu Antônio é o ofertante que sustenta a economia de créditos, uma interface que o exclua inviabiliza o produto inteiro, mesmo que agrade a todos os demais. A acessibilidade, neste projeto, não é requisito de conformidade: é requisito de viabilidade.

5.2.1 Princípios de design adotados

Princípio	Aplicação concreta	Origem
Uma ação por tela	Nenhuma tela do fluxo principal apresenta mais de uma decisão. O cadastro foi dividido em um único passo com três campos.	Persona Antônio; RNF07
Alvos generosos	Nenhum elemento tocável menor que 48 por 48 dp; botões primários com 56 dp de altura.	Material Design; WCAG 2.1
Texto escalável	Todos os textos respeitam a configuração de fonte do sistema e permanecem legíveis com ampliação de 200%, sem truncamento.	WCAG 2.1 AA
Contraste verificado	Razão mínima de 4,5:1 para texto comum e 3:1 para texto grande, validada no Figma antes da implementação.	WCAG 2.1 AA
Estado sempre visível	O saldo de Elos aparece permanentemente na tela inicial; toda troca exibe seu status em linguagem corrente, não em código.	Heurística de visibilidade do estado
Reconhecer em vez de lembrar	A busca oferece categorias navegáveis além do campo de texto, para quem não sabe como nomear o que procura.	Heurística de Nielsen
Nenhum jargão	Nenhuma tela usa termo técnico. "Elos" é explicado na primeira abertura com uma frase e um exemplo.	Persona Antônio

5.2.2 Identidade visual

Elemento	Definição
Cor primária	Verde-azulado #1D9E75 — associado a troca e comunidade, com contraste suficiente sobre branco para texto grande e ícones
Cor de apoio	Âmbar #BA7517 — usada apenas em saldo e créditos, criando associação consistente
Cor de alerta	Vermelho #A32D2D — restrita a erro, denúncia e saldo negativo
Neutros	Cinza #2C2C2A para texto principal, #5F5E5A para secundário, #F1EFE8 para fundos de cartão
Tipografia	Inter, por ter altura de x generosa e boa legibilidade em tamanhos pequenos. Corpo em 16 sp, nunca abaixo de 14 sp
Espaçamento	Escala de 4 em 4 dp; respiro mínimo de 16 dp nas laterais de todas as telas
Raio de borda	12 dp em cartões, 8 dp em campos e botões
Iconografia	Ícones sempre acompanhados de rótulo em texto, nunca isolados

5.2.3 Inventário de telas

O protótipo contempla 16 telas, das quais 12 integram a Onda 1 do sequenciador.

#	Tela	Função	Onda
T01	Splash	Verifica sessão ativa e direciona	1
T02	Login	Autenticação por e-mail e senha	1

#	Tela	Função	Onda
T03	Cadastro	Nome, unidade e código do condomínio	1
T04	Aguardando aprovação	Explica a espera e o que acontece depois	1
T05	Início	Saldo, avisos do síndico e atalhos para buscar e ofertar	1
T06	Buscar	Campo de texto, categorias navegáveis e lista de resultados	1
T07	Perfil do vizinho	Habilidades, reputação e botão de solicitar	1
T08	Solicitar troca	Descrição da necessidade, duração e custo em Elos	1
T09	Minhas trocas	Pedidos recebidos, enviados e em andamento	1
T10	Detalhe da troca	Status, dados do encontro e ações disponíveis	1
T11	Chat	Conversa vinculada à troca, com agendamento no topo	1
T12	Avaliar	Nota de 1 a 5 e comentário opcional	1
T13	Meu perfil	Habilidades cadastradas, saldo e extrato	1
T14	Cadastrar habilidade	Título, categoria, descrição e custo	1
T15	Extrato	Movimentações de Elos com saldo após cada lançamento	2
T16	Painel do síndico (web)	Aprovações, avisos, bloqueios, ranking e lacunas	2

5.2.4 Fluxo de navegação

A navegação principal usa barra inferior com três destinos: Início, Buscar e Minhas trocas. O perfil fica acessível pelo avatar no canto superior. A escolha pela barra inferior em vez de menu lateral decorre diretamente da persona de baixa afinidade digital: um menu escondido atrás de um ícone é exatamente o padrão que faz o Seu Antônio desistir.

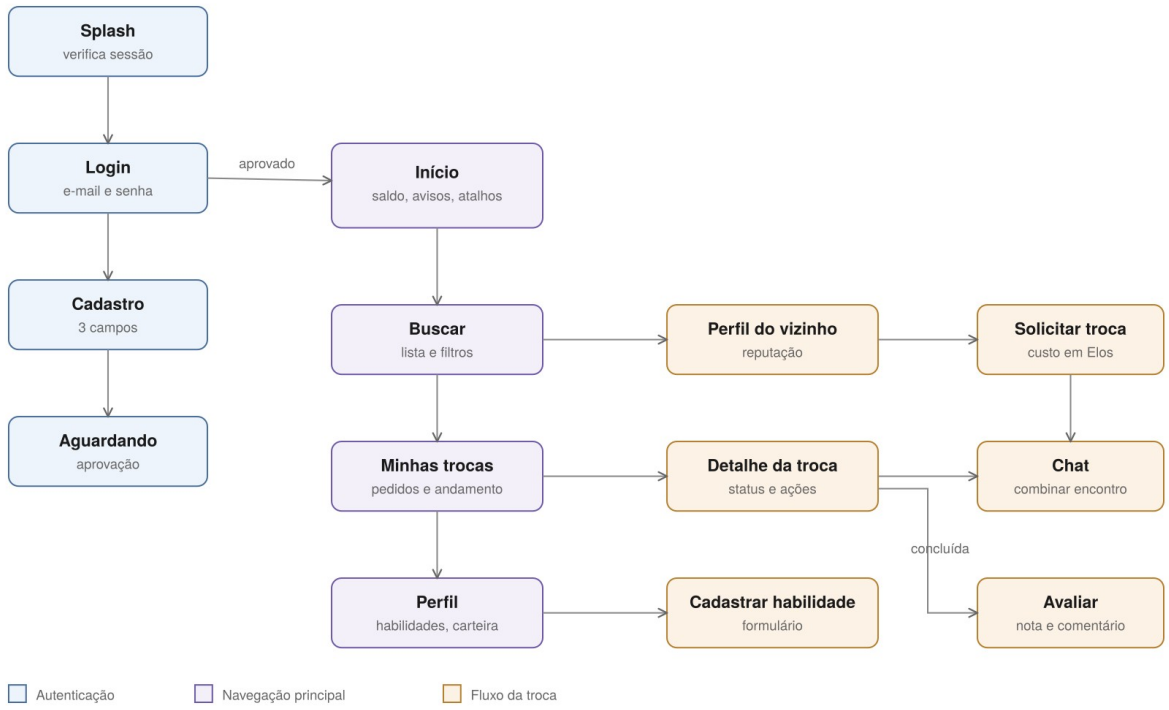


Figura 2 — Fluxo de navegação do aplicativo

5.2.5 Decisões de usabilidade por persona

Dor da persona	Decisão de interface
Camila desconfia de app que pede telefone e endereço no cadastro	O cadastro pede apenas nome, unidade e código. Telefone e e-mail nunca são exibidos a outros moradores; o contato acontece só pelo chat interno
Camila acha que não tem nada a oferecer	Ao concluir o cadastro, o app sugere habilidades em falta no prédio e pergunta se alguma delas se aplica, em vez de apresentar um campo em branco
Camila teme deixar estranho entrar em casa	A tela de agendamento sugere apenas áreas comuns, nunca o apartamento, e exibe a reputação do vizinho ao lado do local
Antônio abandona cadastros longos	Cadastro em uma tela com três campos; tutorial pode ser pulado com um toque
Antônio tem dificuldade com botões pequenos	Botões primários de 56 dp e modo de fonte ampliada acessível na primeira execução, não escondido em configurações
Antônio desconfia de golpe	Nenhuma tela menciona dinheiro, cartão ou pagamento. A primeira abertura explica em uma frase que o app não envolve dinheiro real
Antônio sente que ficou invisível	O perfil exibe com destaque o número de trocas concluídas e os selos recebidos
Rita não tem tempo para mais um canal	O painel web abre direto no que exige ação: cadastros pendentes e denúncias. Os relatórios ficam em segundo nível

5.2.6 Protótipo navegável

O protótipo de alta fidelidade reproduz o fluxo de navegação apresentado acima. As seis telas abaixo cobrem o caminho completo de uma troca, da descoberta à conclusão, e foram desenhadas aplicando os princípios da seção 5.2.1.

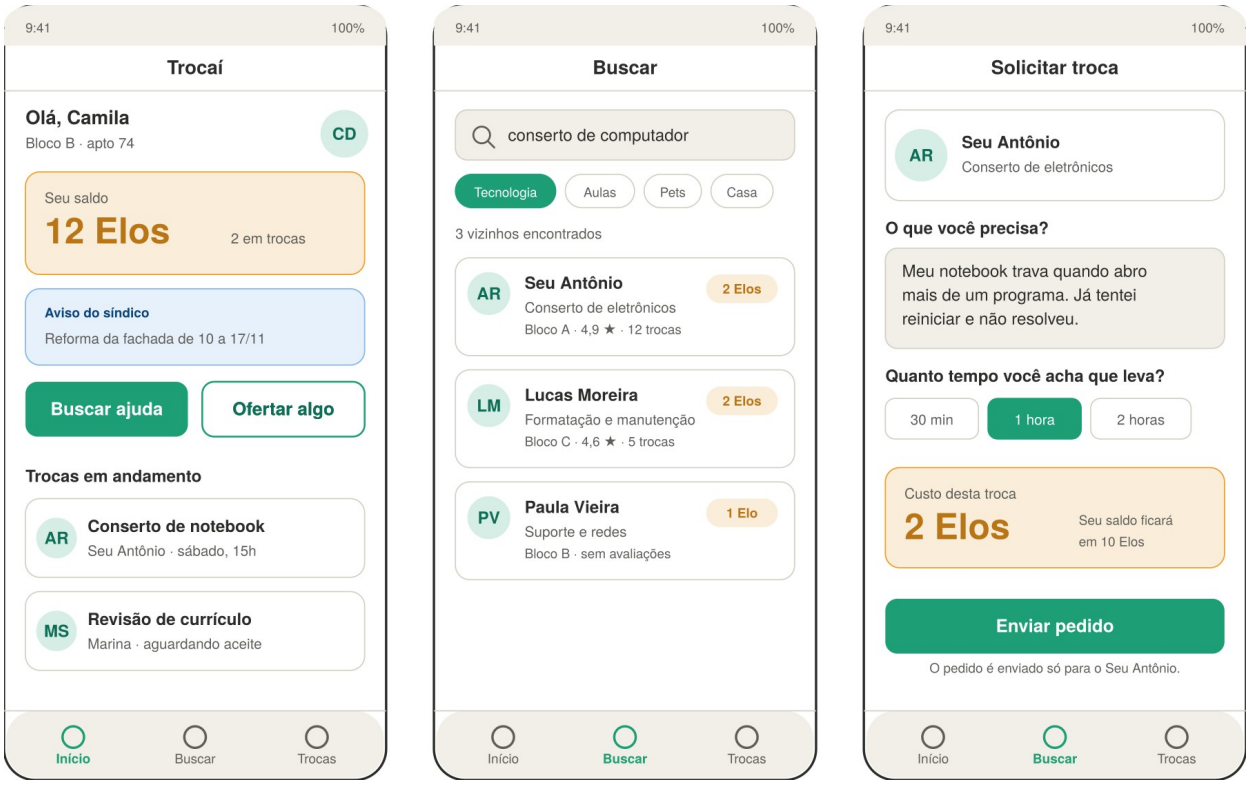


Figura 3 — Telas T05 Início, T06 Buscar e T08 Solicitar troca

Na tela de início, o saldo ocupa a posição de maior destaque porque é a informação que responde à pergunta "posso pedir alguma coisa?". O aviso do síndico aparece logo abaixo, substituindo o cartaz do

elevador. Os dois botões de ação usam verbos na linguagem do morador — buscar ajuda e ofertar algo — em vez de termos de produto.

A busca oferece categorias navegáveis ao lado do campo de texto, atendendo quem não sabe como nomear o que procura. Cada resultado mostra bloco, avaliação e número de trocas, que são os três dados que sustentam a decisão de confiar. Quem ainda não tem avaliação recebe o rótulo explícito "sem avaliações", em vez de ser empurrado para o fim da lista.

Na solicitação, o custo em Elos aparece antes do botão de envio, junto com a projeção do saldo depois da operação. A duração é escolhida entre três opções em vez de digitada, o que elimina teclado numérico do fluxo principal.

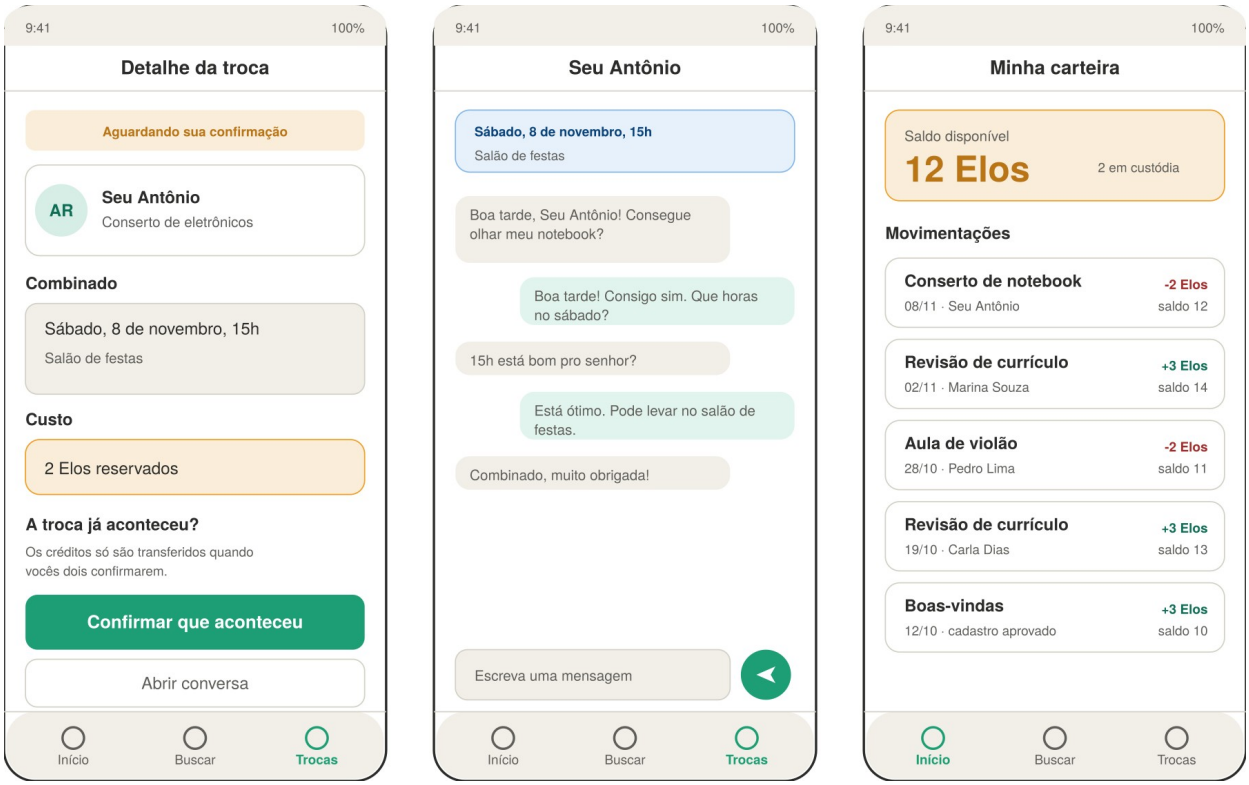


Figura 4 — Telas T10 Detalhe da troca, T11 Chat e T15 Carteira

O detalhe da troca explica, em texto corrido, por que a confirmação do outro é necessária. Essa frase existe porque a dupla confirmação é a regra menos intuitiva do produto e a que mais gera dúvida na primeira troca.

O chat fixa data e local no topo, de modo que a informação combinada não se perca no histórico. A carteira mostra o saldo disponível separado do que está em custódia, e cada lançamento exibe o saldo resultante, o que permite ao morador conferir a conta sem fazer aritmética.

A validação de contraste foi executada sobre a paleta definida em 5.2.2. Todas as combinações de texto sobre fundo em uso nas telas atendem o nível AA da WCAG 2.1: texto principal em #2C2C2A sobre branco alcança razão de 14,9:1 e o texto secundário em #5F5E5A alcança 6,9:1, ambos acima do mínimo de 4,5:1.

5.3 Estrutura do Banco de Dados

A escolha pelo PostgreSQL, em vez de uma base NoSQL gerenciada, decorre de um único requisito: a carteira de Elos não pode divergir em hipótese alguma. Uma transferência de créditos precisa debitar um morador e creditar outro como operação indivisível; se metade da operação for gravada e a outra não, o sistema passa a mostrar saldos que não existem, e a confiança na economia interna termina. Bancos relacionais oferecem transações ACID nativamente, e essa garantia pesou mais que a facilidade de configuração de alternativas como o Firestore.

5.3.1 Modelo conceitual

O domínio se organiza em torno de três conceitos. O condomínio é a fronteira de isolamento: nenhum dado atravessa condomínios, o que também prepara o modelo para operação multi-tenant. O morador é simultaneamente ofertante e solicitante, papéis que não são atributos da pessoa mas da troca em que ela participa. A troca é a entidade central, que conecta duas pessoas, uma habilidade, um encontro e duas movimentações de crédito.

Três decisões de modelagem merecem registro. Primeira: o saldo do morador é um campo materializado em morador.saldo_elos, e não uma soma calculada sobre movimentacao_elos a cada consulta, porque o saldo é lido em toda abertura do aplicativo; a consistência entre os dois é garantida por transação e verificada por rotina de auditoria. Segunda: as confirmações das duas partes ficam como colunas na própria troca, e não em tabela separada, porque são sempre exatamente duas e nunca mais. Terceira: movimentacao_elos é um livro-razão imutável — não há atualização nem exclusão, apenas inserção, o que permite reconstruir qualquer saldo histórico.

5.3.2 Modelo entidade-relacionamento

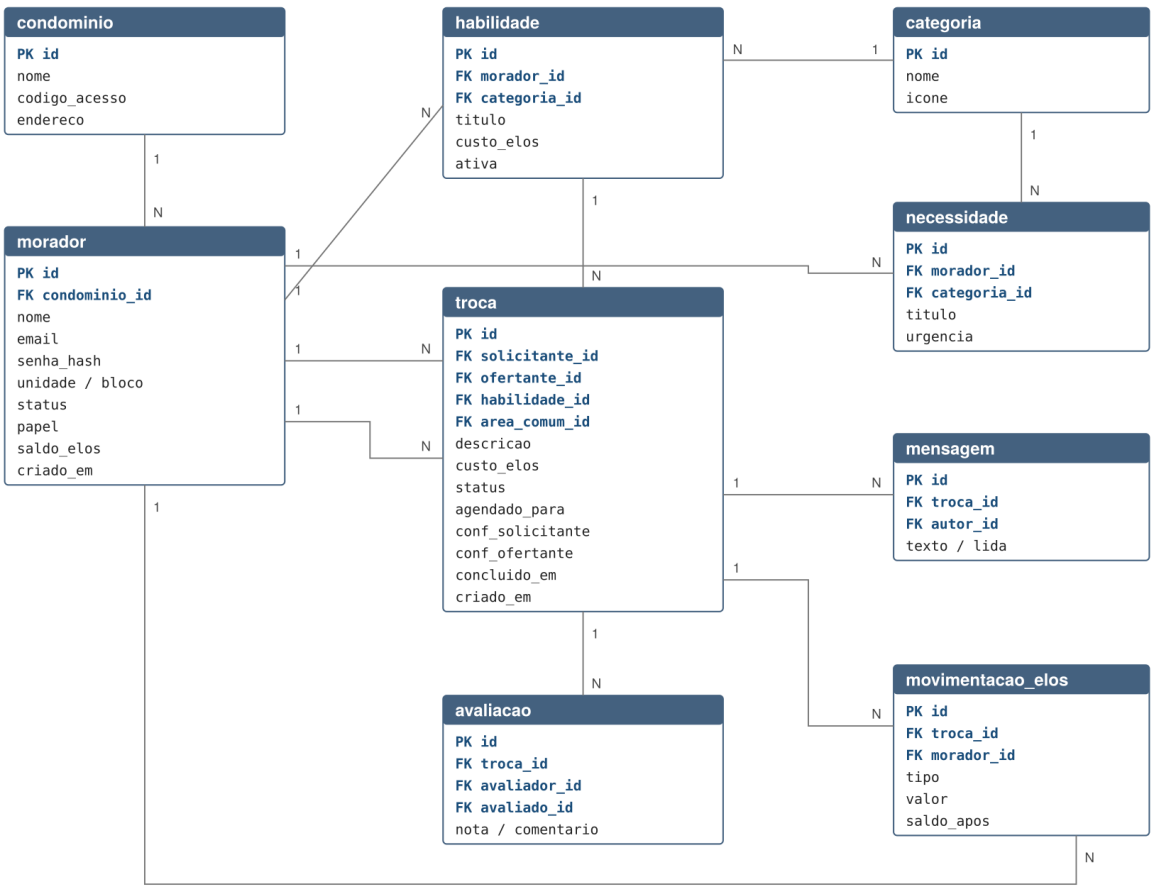


Figura 5 — Modelo entidade-relacionamento do Trocaí

5.3.3 Dicionário de dados

Tabela morador

Coluna	Tipo	Restrição	Descrição
id	BIGSERIAL	PK	Identificador do morador
condominio_id	BIGINT	FK, NOT NULL	Condomínio ao qual pertence
nome	VARCHAR(120)	NOT NULL	Nome exibido publicamente
email	VARCHAR(160)	UNIQUE, NOT NULL	Usado só para login; nunca exibido a outros
senha_hash	VARCHAR(255)	NOT NULL	Hash bcrypt da senha
unidade	VARCHAR(10)	NOT NULL	Número da unidade; não exibido publicamente
bloco	VARCHAR(10)	NULL	Bloco ou torre; exibido publicamente
status	VARCHAR(20)	NOT NULL, DEFAULT 'pendente'	pendente, ativo, suspenso ou recusado
papel	VARCHAR(20)	NOT NULL, DEFAULT 'morador'	morador ou sindico
saldo_elos	INTEGER	NOT NULL, DEFAULT 0	Saldo disponível; não inclui créditos em custódia
elos_custodia	INTEGER	NOT NULL, DEFAULT 0	Créditos reservados em trocas aceitas
criado_em	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Data do cadastro

Tabela troca

Coluna	Tipo	Restrição	Descrição
id	BIGSERIAL	PK	Identificador da troca
solicitante_id	BIGINT	FK morador, NOT NULL	Quem pediu
ofertante_id	BIGINT	FK morador, NOT NULL	Quem oferece a habilidade
habilidade_id	BIGINT	FK habilidade, NOT NULL	Habilidade solicitada
area_comum_id	BIGINT	FK area_comum, NULL	Local combinado do encontro
descricao	TEXT	NOT NULL	Descrição da necessidade escrita pelo solicitante
duracao_estimada	INTEGER	NULL	Minutos estimados
custo_elos	INTEGER	NOT NULL, CHECK > 0	Congelado no momento do aceite
status	VARCHAR(20)	NOT NULL	pendente, aceita, concluida, cancelada ou mediacao
agendado_para	TIMESTAMPTZ	NULL	Data e hora combinadas
conf_solicitante	TIMESTAMPTZ	NULL	Momento da confirmação do solicitante
conf_ofertante	TIMESTAMPTZ	NULL	Momento da confirmação do ofertante
concluido_em	TIMESTAMPTZ	NULL	Preenchido quando ambas confirmam
criado_em	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Data da solicitação

Tabela movimentacao_elos — livro-razão imutável

Coluna	Tipo	Restrição	Descrição
id	BIGSERIAL	PK	Identificador do lançamento
troca_id	BIGINT	FK troca, NULL	Troca que originou; nulo em bônus de boas-vindas
morador_id	BIGINT	FK morador, NOT NULL	Morador afetado
tipo	VARCHAR(20)	NOT NULL	credito, debito, custodia, devolucao ou bonus
valor	INTEGER	NOT NULL, CHECK <> 0	Positivo em entrada, negativo em saída
saldo_apos	INTEGER	NOT NULL	Saldo do morador após o lançamento
criado_em	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Momento do lançamento

As demais tabelas — condominio, categoria, habilidade, necessidade, avaliacao, mensagem, aviso, bloqueio, denuncia e area_comum — seguem a mesma convenção: chave primária BIGSERIAL, carimbo criado_em obrigatório e chaves estrangeiras com integridade referencial declarada.

5.3.4 Regras de integridade da carteira

As regras abaixo são aplicadas no banco, e não apenas na aplicação, para que nenhuma falha de código consiga corromper os saldos.

#	Regra	Implementação
I1	O saldo nunca fica abaixo de -3 Elos	CHECK (saldo_elos >= -3) na tabela morador
I2	Créditos em custódia nunca são negativos	CHECK (elos_custodia >= 0)
I3	Toda troca tem custo positivo	CHECK (custo_elos > 0)

#	Regra	Implementação
I4	Ninguém troca consigo mesmo	CHECK (solicitante_id <> ofertante_id)
I5	Uma troca gera no máximo uma avaliação por participante	UNIQUE (troca_id, avaliador_id)
I6	A soma das movimentações de um morador é igual ao seu saldo	Rotina de auditoria diária que compara SUM(valor) com saldo_elos e registra divergências
I7	Lançamentos não podem ser alterados nem apagados	Revogação de UPDATE e DELETE em movimentacao_elos para o usuário da aplicação
I8	A transferência é tudo ou nada	Bloco BEGIN / COMMIT com SELECT FOR UPDATE nas duas linhas de morador

5.3.5 Script de criação — trecho principal

```
CREATE TABLE morador (
  id              BIGSERIAL PRIMARY KEY,
  condominio_id   BIGINT NOT NULL REFERENCES condominio(id),
  nome            VARCHAR(120) NOT NULL,
  email           VARCHAR(160) NOT NULL UNIQUE,
  senha_hash      VARCHAR(255) NOT NULL,
  unidade         VARCHAR(10) NOT NULL,
  bloco           VARCHAR(10),
  status          VARCHAR(20) NOT NULL DEFAULT 'pendente',
  papel           VARCHAR(20) NOT NULL DEFAULT 'morador',
  saldo_elos      INTEGER NOT NULL DEFAULT 0,
  elos_custodia   INTEGER NOT NULL DEFAULT 0,
  criado_em       TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT saldo_minimo   CHECK (saldo_elos >= -3),
  CONSTRAINT custodia_valida CHECK (elos_custodia >= 0)
);

CREATE TABLE troca (
  id                BIGSERIAL PRIMARY KEY,
  solicitante_id    BIGINT NOT NULL REFERENCES morador(id),
  ofertante_id      BIGINT NOT NULL REFERENCES morador(id),
  habilidade_id     BIGINT NOT NULL REFERENCES habilidade(id),
  area_comum_id     BIGINT REFERENCES area_comum(id),
  descricao         TEXT NOT NULL,
  duracao_estimada  INTEGER,
  custo_elos        INTEGER NOT NULL,
  status            VARCHAR(20) NOT NULL DEFAULT 'pendente',
  agendado_para     TIMESTAMPTZ,
  conf_solicitante  TIMESTAMPTZ,
  conf_ofertante    TIMESTAMPTZ,
  concluido_em      TIMESTAMPTZ,
  criado_em         TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT custo_positivo CHECK (custo_elos > 0),
  CONSTRAINT partes_distintas CHECK (solicitante_id <> ofertante_id)
);

CREATE INDEX idx_habilidade_busca
  ON habilidade USING gin (to_tsvector('portuguese', titulo || ' ' || descricao));
CREATE INDEX idx_troca_ofertante ON troca (ofertante_id, status);
CREATE INDEX idx_mov_morador ON movimentacao_elos (morador_id, criado_em DESC);
```

5.3.6 Estratégia transacional da transferência

A operação crítica do sistema é a transferência de Elos na conclusão da troca. Ela é executada em transação única, com travamento explícito das linhas envolvidas e em ordem determinística de identificador, para evitar impasse quando duas trocas entre as mesmas pessoas são concluídas ao mesmo tempo.

```
BEGIN;

-- trava as duas linhas sempre na mesma ordem para evitar deadlock
SELECT id, saldo_elos, elos_custodia FROM morador
  WHERE id IN ($solicitante, $ofertante)
  ORDER BY id
  FOR UPDATE;

-- libera a custódia e credita o ofertante
UPDATE morador SET elos_custodia = elos_custodia - $custo
  WHERE id = $solicitante;
UPDATE morador SET saldo_elos = saldo_elos + $custo
  WHERE id = $ofertante;

-- livro-razão: duas linhas, nunca uma só
INSERT INTO movimentacao_elos (troca_id, morador_id, tipo, valor, saldo_apos)
  VALUES ($troca, $solicitante, 'debito', -$custo, $saldo_sol),
         ($troca, $ofertante,   'credito', $custo, $saldo_ofe);

UPDATE troca SET status = 'concluida', concluido_em = now()
  WHERE id = $troca;

COMMIT;
```

Qualquer falha dentro do bloco provoca ROLLBACK automático, e nenhum saldo é alterado. A restrição de verificação saldo_minimo atua como última barreira: mesmo que um defeito na aplicação tente levar o saldo abaixo de -3, o banco recusa a operação e a transação inteira é revertida.

5.4 Implementação

A implementação cobre as funcionalidades da Onda 1 definida no sequenciador, contemplando as cinco áreas previstas para esta entrega: cadastro de morador com validação pelo síndico, publicação de habilidades, busca com filtros, solicitação de troca e movimentação de créditos. Acompanham o núcleo o perfil público, o registro de necessidades, o aceite, a conclusão com dupla confirmação, a avaliação e o chat simples.

5.4.1 Arquitetura da solução

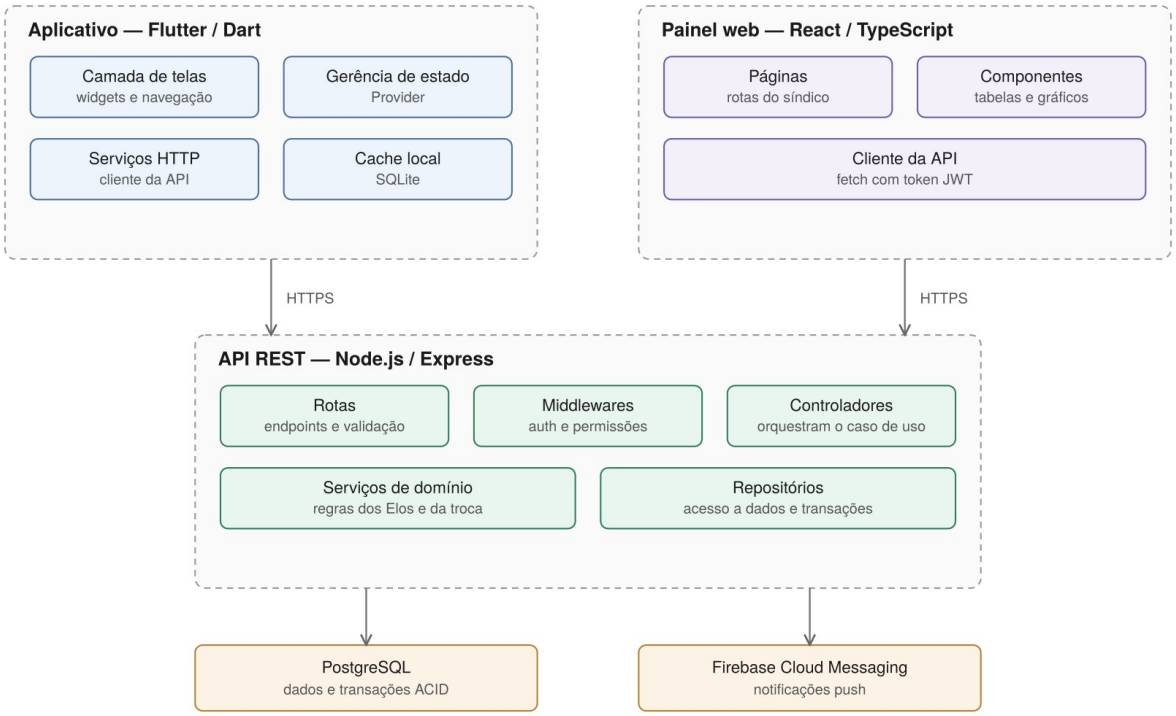


Figura 6 — Arquitetura em camadas do Trocaí

A API concentra toda a regra de negócio. O aplicativo e o painel web são clientes sem lógica de domínio, o que garante que as regras dos Elos sejam idênticas nas duas interfaces e que uma eventual terceira interface não precise reimplementá-las. A separação entre serviços de domínio e repositórios mantém as regras testáveis sem banco de dados.

5.4.2 Organização do código

```
trocai-api/
  src/
    rotas/           definição dos endpoints e validação de entrada
    middlewares/     autenticação JWT, permissões, tratamento de erro
    controladores/   orquestram um caso de uso por método
    servicos/        regras de negócio (TrocaService, CarteiraService)
    repositorios/    acesso ao banco e controle de transação
    db/              pool de conexões e migrações
  testes/
    unitarios/       serviços de domínio, sem banco
    integracao/      endpoints contra banco de teste

trocai-app/
  lib/
```

```
telas/          uma pasta por tela do inventário de telas
componentes/    widgets reutilizáveis do design system
estado/         providers
servicos/       cliente HTTP e armazenamento local
modelos/        classes de domínio
```

5.4.3 Endpoints da API

São 32 endpoints sob o prefixo /api. Fora /saude, /auth/cadastro e /auth/login, toda rota exige o cabeçalho Authorization com o token da sessão. As rotas sob /admin exigem papel de síndico, verificado no próprio token.

Método e rota	Função	Caso de uso
GET /saude	Verificação de disponibilidade	—
POST /auth/cadastro	Cria conta pendente de aprovação	UC01
POST /auth/login	Autentica e devolve token JWT	—
GET /auth/eu	Dados do morador autenticado	—
GET /categorias	Categorias para o filtro da busca	UC04
GET /moradores/:id	Perfil público do vizinho	UC04
GET /habilidades?q=&categoria=	Busca com filtros	UC04
GET /habilidades/minhas	Habilidades que eu publiquei	UC02
POST /habilidades	Publica habilidade oferecida	UC02
DELETE /habilidades/:id	Despublica uma habilidade	UC02
GET /necessidades	O que o condomínio procura	UC03
POST /necessidades	Registra necessidade procurada	UC03
GET /trocas?papel=&status=	Lista trocas do morador autenticado	UC06
POST /trocas	Solicita troca a um vizinho	UC05
PATCH /trocas/:id/aceitar	Aceita e reserva os créditos	UC06
PATCH /trocas/:id/recusar	Recusa sem movimentar créditos	UC06
PATCH /trocas/:id/confirmar	Registra confirmação de uma das partes	UC07
PATCH /trocas/:id/cancelar	Cancela e devolve o que estava retido	UC06
GET /trocas/:id/mensagens	Lê o chat da troca	UC08
POST /trocas/:id/mensagens	Envia mensagem no chat da troca	UC08
POST /trocas/:id/avaliacao	Avalia a outra parte	UC09
POST /denuncias	Registra denúncia de conduta	UC16
GET /carteira	Saldo, custódia e extrato	UC10
GET /carteira/auditoria	Confere o saldo contra o livro-razão	UC10
GET /admin/cadastros	Lista cadastros pendentes	UC11
PATCH /admin/cadastros/:id	Aprova ou recusa cadastro	UC11
PATCH /admin/moradores/:id/suspender	Suspende o acesso de um morador	UC12
GET /admin/painel	Indicadores, ranking e Árvore de Talentos	UC14

Método e rota	Função	Caso de uso
POST /admin/bloqueios	Suspende as trocas do condomínio	UC13
PATCH /admin/bloqueios/:id/liberar	Encerra a suspensão	UC13
GET /admin/denuncias	Denúncias em aberto	UC16
PATCH /admin/denuncias/:id	Registra a decisão do síndico	UC15, UC16

Os códigos de resposta seguem um padrão único: 401 para token ausente, inválido, cadastro pendente, acesso suspenso, falta de permissão ou operação fora das próprias trocas; 404 para recurso inexistente; 409 para registro duplicado; 422 para regra de negócio recusada; e 500 para falha interna, cujo detalhe fica no log e não na resposta. Dentro de /api, uma rota inexistente responde 401 a quem não apresentou token, antes de dizer se ela existe: responder 404 permitiria mapear a API de fora.

5.4.4 Implementação da regra crítica

O método abaixo concentra o maior risco técnico do sistema. A confirmação só dispara a transferência quando as duas partes já confirmaram, e toda a operação acontece dentro de uma transação.

```
async confirmarConclusao(trocaId, moradorId) {
  return this.db.transacao(async (tx) => {
    const troca = await tx.trocas.buscarParaAtualizar(trocaId);

    if (troca.status !== 'aceita')
      throw new RegraDeNegocio('A troca não está em andamento.');

    const ehSolicitante = troca.solicitante_id === moradorId;
    const ehOfertante   = troca.ofertante_id === moradorId;
    if (!ehSolicitante && !ehOfertante)
      throw new NaoAutorizado('Você não participa desta troca.');

    await tx.trocas.registrarConfirmacao(trocaId, ehSolicitante);

    const atual = await tx.trocas.buscar(trocaId);
    const ambasConfirmaram = atual.conf_solicitante && atual.conf_ofertante;

    if (!ambasConfirmaram) {
      return { status: 'aguardando_outra_parte' };
    }

    // dupla confirmação: transfere os créditos
    await this.carteira.transferir(tx, {
      de: atual.solicitante_id,
      para: atual.ofertante_id,
      valor: atual.custo_elos,
      trocaId
    });

    await tx.trocas.concluir(trocaId);
    return { status: 'concluida' };
  });
}
```

Qualquer exceção lançada dentro do bloco reverte a transação inteira, incluindo a confirmação já registrada. Isso evita o estado inconsistente em que uma confirmação é gravada mas a transferência falha.

5.4.5 Telas implementadas

As telas da Onda 1 foram implementadas conforme o protótipo de interface definido na entrega anterior. A sequência abaixo mostra o fluxo completo de uma troca em execução, com dados reais vindos da API.

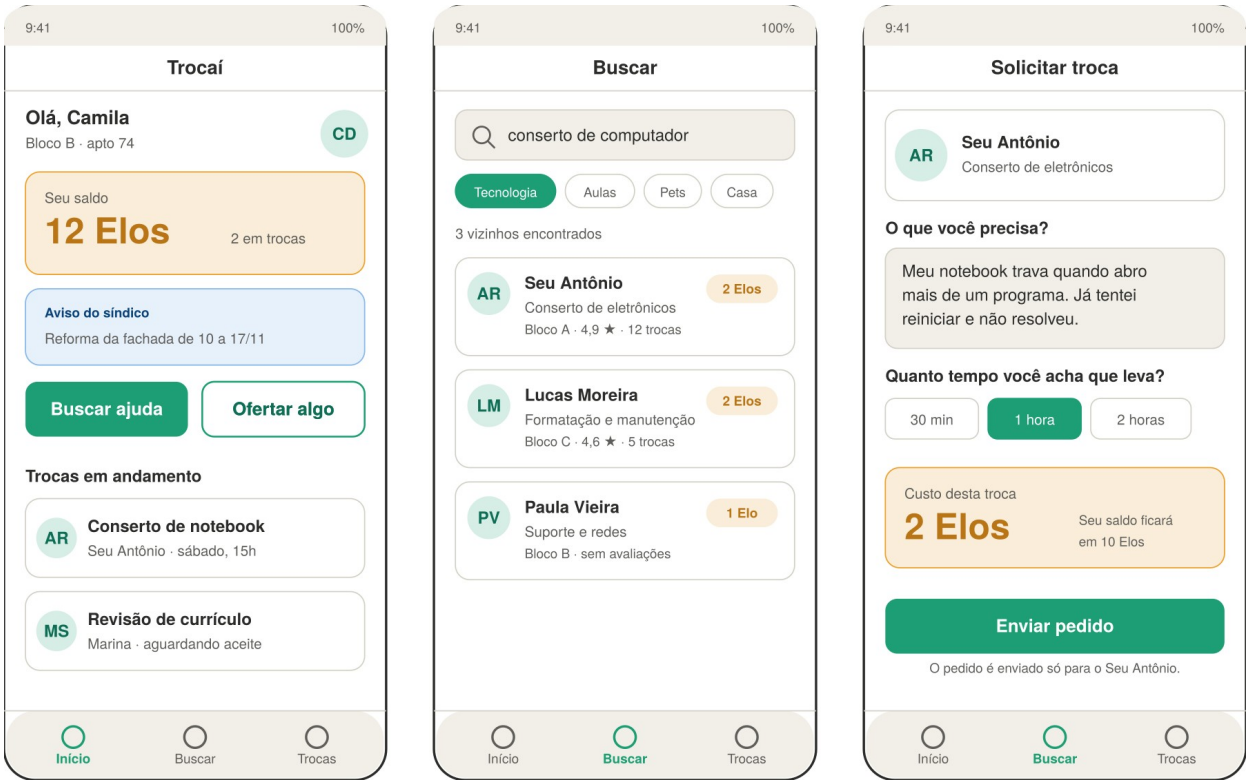


Figura 7 — Início com saldo carregado da API, busca com resultados e tela de solicitação

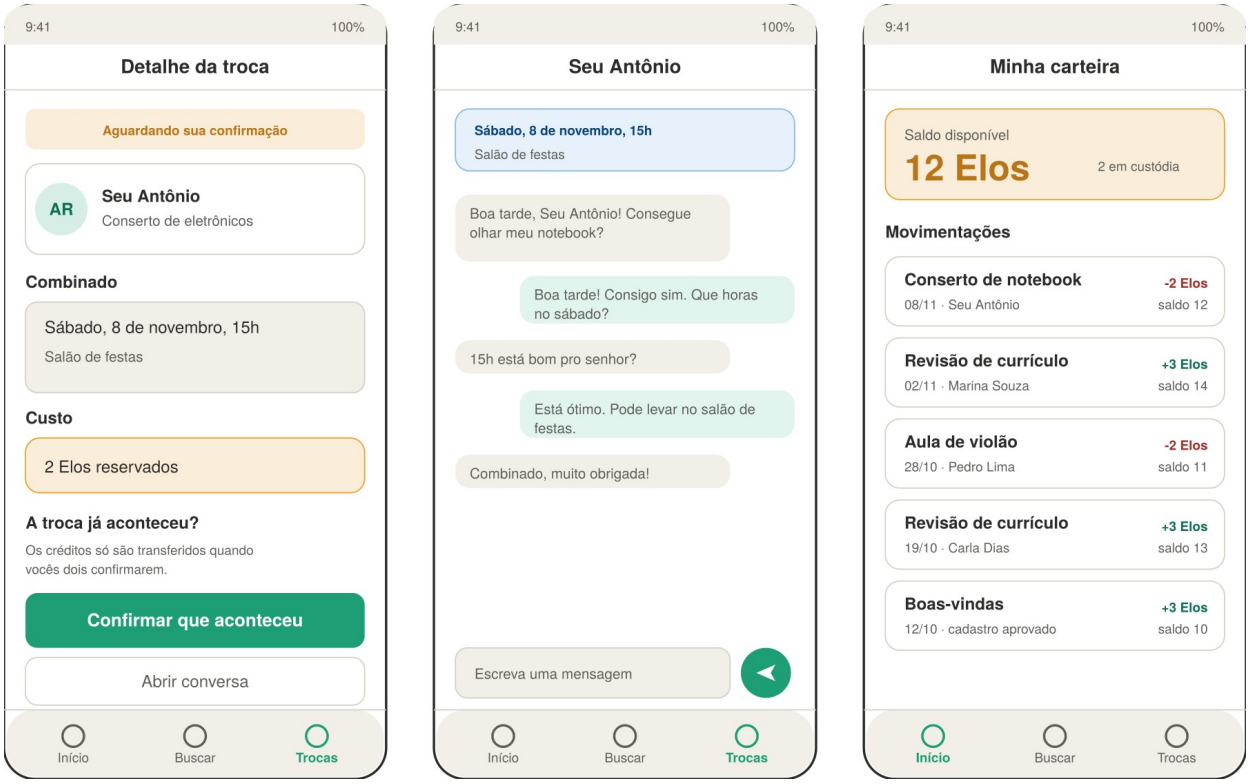


Figura 8 — Troca aguardando confirmação, chat da troca e extrato após a transferência

O extrato da terceira tela da última figura corresponde ao estado do banco após a execução do caso CT03: a linha de 8 de novembro registra o débito de 2 Elos e o saldo resultante de 12, exatamente como gravado na tabela movimentacao_elos.

5.4.6 Decisões técnicas registradas

Decisão	Alternativa descartada	Motivo
Regra de negócio apenas na API	Validação duplicada no aplicativo	Garante que app e painel web apliquem a mesma regra; o cliente valida só formato de campo
Chat por requisição periódica na Onda 1	WebSocket desde o início	A versão simples entrega quase todo o valor de experiência com conhecimento técnico verde, conforme a revisão da Atividade 7
Custo em Elos congelado no aceite	Custo lido da habilidade na conclusão	Impede que o ofertante altere o preço depois do combinado
Saldo materializado em coluna	Soma calculada a cada leitura	O saldo é lido em toda abertura do app; a consistência é garantida por transação e auditoria
Busca com índice de texto do PostgreSQL	Serviço de busca dedicado	O volume de um condomínio não justifica infraestrutura adicional

5.4.7 Repositório do código-fonte

O código-fonte do produto está publicado em:

```
https://github.com/sarahquinteiro/trocai
```

O repositório reúne, além do código, a documentação de todas as entregas e os artefatos visuais do projeto:

Caminho	Conteúdo
api/src/	Implementação da API: rotas, middlewares, controladores, serviços, repositórios e migrações
api/testes/	Suíte automatizada: testes unitários de domínio e testes de integração dos endpoints
api/scripts/	Script que percorre uma troca inteira pela API, para demonstração
api/README.md	Endpoints, códigos de resposta e decisões técnicas
docs/entregas/	Os documentos das seis entregas em PDF
docs/lean-inception/	Registro das nove atividades do workshop
docs/diagramas/	Casos de uso, navegação, modelo de dados, arquitetura e telas

Para executar:

```
cd api
npm install
cp .env.example .env        # credenciais do PostgreSQL
createdb trocai
npm run migrar              # cria o esquema e os dados de exemplo
npm start                   # http://localhost:3000/api

npm test                    # suíte automatizada completa
npm run demonstracao        # troca inteira, sem precisar de banco
```

O comando npm run demonstracao sobe a API sobre o banco de teste em memória e executa o fluxo completo — cadastro, aprovação pelo síndico, busca, pedido, aceite, conversa, dupla confirmação, extrato, auditoria de integridade e avaliação —, imprimindo o que acontece com os saldos das duas carteiras a cada passo. É a forma de verificar o comportamento do domínio em qualquer máquina, sem instalar PostgreSQL. Os resultados da suíte automatizada estão registrados na seção de testes e validação.

5.5 Testes e Validação

A estratégia de testes foi desenhada em torno do risco identificado na revisão técnica: a integridade da carteira de Elos. Um defeito na busca gera incômodo; um defeito na transferência de créditos destrói a confiança no produto e não tem recuperação possível. Por isso a cobertura automatizada se concentra nos serviços de domínio, e os casos de teste funcionais cobrem com mais profundidade os fluxos de troca.

5.5.1 Níveis de teste

Nível	Escopo	Ferramenta	Critério de saída
Unitário	Serviços de domínio sem banco: cálculo de custo, limite de dívida, transição de status	Jest	Cobertura mínima de 60% conforme RNF11
Integração	Endpoints da API contra banco de teste, incluindo transações e reversões	Jest e Supertest	Todos os endpoints da Onda 1 com teste de caminho feliz e de exceção
Funcional	Fluxos completos executados manualmente conforme casos documentados em 5.5.2	Execução manual registrada	Nenhum defeito crítico em aberto
Usabilidade	Validação com moradores reais, com atenção ao perfil de baixa afinidade digital	Observação assistida	Pelo menos um participante do perfil Antônio conclui o fluxo sem ajuda

5.5.2 Casos de teste funcionais

Os casos seguem a estrutura de identificação, pré-condição, passos, resultado esperado e resultado obtido. Os cinco abaixo cobrem o núcleo; a suíte completa consta no apêndice de testes.

CT01 — Cadastro com código de condomínio inválido

Campo	Conteúdo
Pré-condição	Nenhuma sessão ativa
Passos	1. Abrir a tela de cadastro. 2. Preencher nome e unidade. 3. Informar código inexistente. 4. Enviar.
Resultado esperado	O sistema exibe mensagem de código inválido, mantém os dados preenchidos e não cria a conta.
Resultado obtido	

CT02 — Solicitação bloqueada por limite de dívida

Campo	Conteúdo
Pré-condição	Morador autenticado com saldo igual a -3 Elos
Passos	1. Buscar uma habilidade. 2. Abrir o perfil do ofertante. 3. Tocar em solicitar troca.
Resultado esperado	O sistema bloqueia a ação e explica que é preciso ofertar antes de voltar a pedir. Nenhuma solicitação é criada.
Resultado obtido	

CT03 — Transferência só ocorre após a dupla confirmação

Campo	Conteúdo
Pré-condição	Troca com status aceita, solicitante com 5 Elos e ofertante com 2 Elos, custo de 2 Elos
Passos	1. O solicitante confirma a conclusão. 2. Consultar os saldos. 3. O ofertante confirma. 4. Consultar os

Campo	Conteúdo
	saldos novamente.
Resultado esperado	Após o passo 2 nenhum saldo muda e a troca fica aguardando a outra parte. Após o passo 4 o solicitante fica com 3 Elos, o ofertante com 4, a troca passa a concluída e duas linhas são gravadas no extrato.
Resultado obtido	

CT04 — Reversão da transação em caso de falha

Campo	Conteúdo
Pré-condição	Troca aceita e ambiente de teste com falha forçada após o débito
Passos	1. Provocar a dupla confirmação com interrupção simulada antes do crédito ao ofertante. 2. Consultar os saldos e o extrato.
Resultado esperado	Nenhum saldo é alterado, nenhuma linha é gravada no extrato e a troca permanece com status aceita.
Resultado obtido	

CT05 — Bloqueio de período preserva trocas já aceitas

Campo	Conteúdo
Pré-condição	Uma troca aceita e um bloqueio cadastrado pelo síndico para o período atual
Passos	1. Tentar criar nova solicitação. 2. Abrir a troca já aceita e confirmar a conclusão.
Resultado esperado	A nova solicitação é recusada com aviso do período bloqueado. A troca já aceita conclui normalmente e os créditos são transferidos.
Resultado obtido	

5.5.3 Protocolo de validação com usuários

A validação é conduzida com seis moradores de um condomínio piloto, sendo pelo menos dois com baixa afinidade digital, recrutados com o critério de usarem o celular principalmente para mensagens e chamadas. Cada sessão dura cerca de 20 minutos, é individual e segue o roteiro de tarefas abaixo, sem instrução prévia sobre como executá-las.

#	Tarefa proposta ao participante	O que se observa
T1	Crie sua conta no aplicativo	Tempo até concluir; desistência; dúvidas sobre o código do condomínio
T2	Cadastre algo que você sabe fazer	Se compreende o conceito sem explicação; se consegue definir o custo
T3	Encontre alguém no prédio que conserte computador	Se usa busca por texto ou por categoria; se encontra sem ajuda
T4	Peça ajuda a essa pessoa	Se entende o custo em Elos antes de confirmar
T5	Confirme que a troca aconteceu	Se compreende por que precisa da confirmação do outro
T6	Descubra quantos Elos você tem	Se localiza o saldo sem procurar em menus

As métricas coletadas são taxa de conclusão por tarefa, tempo por tarefa, número de pedidos de ajuda e pontuação SUS ao final. O critério de aprovação é taxa de conclusão de pelo menos 80% nas tarefas T1 a T4 e SUS igual ou superior a 68.

5.5.4 Resultados dos testes automatizados

A suíte reúne dois níveis. Os testes unitários exercitam os serviços de domínio isoladamente; os de integração percorrem os endpoints de ponta a ponta, passando por rota, middleware de autenticação, validação, controlador, serviço e repositório. Nos dois casos o banco de teste é uma implementação em memória que reproduz o comportamento transacional do PostgreSQL: em caso de exceção, o estado retorna exatamente ao ponto anterior ao início da transação. Isso permite verificar a reversão sem depender de infraestrutura externa.

Métrica	Resultado	Critério
Suítes executadas	4 de 4 aprovadas	—
Testes executados	76, todos aprovados	Nenhuma falha
Testes unitários	42	—
Testes de integração	34	—
Cobertura de instruções	90,52%	Mínimo de 60% (RNF11)
Cobertura de ramos	78,29%	—
Cobertura de funções	97,50%	—
Cobertura de linhas	92,27%	—
Tempo de execução	1,7 segundo	—

Cobertura por camada

Camada	Instruções	Ramos	Funções	Linhas
src/rotas	100%	100%	100%	100%
src (aplicação, configuração e erros)	94,28%	89,65%	100%	97,05%
src/controladores	94,20%	50,00%	94,73%	95,16%
src/servicos	88,96%	78,71%	100%	91,36%
src/middlewares	82,53%	71,73%	92,85%	83,33%
Total	90,52%	78,29%	97,50%	92,27%

Dentro dos serviços, os dois arquivos de maior risco são os que guardam a regra dos Elos.

Arquivo	Instruções	Ramos	Funções	Linhas
src/servicos/carteira.js	95,55%	87,50%	100%	95,45%
src/servicos/troca.js	89,02%	79,36%	100%	93,58%

A camada de repositório fica fora da medição. Ela é SQL puro e só executa contra o PostgreSQL, de modo que a suíte automatizada não a alcança: sua verificação está nos casos funcionais executados contra o banco real. Incluí-la na conta produziria um número sem significado, em qualquer direção. As demais linhas não cobertas correspondem a ramos de erro de infraestrutura, como falha de conexão com o banco.

5.5.5 Relação dos testes executados

A listagem abaixo é a saída da execução, agrupada por caso de uso. Os testes marcados com código CT correspondem aos casos funcionais documentados na seção 5.5.2.

```
ARQUIVO: unitarios/carteira.test.js
  CarteiraService — conversão de duração em Elos (RN01)
    OK   30 minutos valem 1 Elo
    OK   1 hora vale 2 Elos
    OK   arredonda para cima: 45 minutos valem 2 Elos
```

```
    OK   duração mínima cobra ao menos 1 Elo
    OK   duração inválida é recusada
  CarteiraService — limite de dívida (RN05)
    OK   permite ficar exatamente no limite de -3
    OK   recusa ultrapassar o limite de -3
    OK   permite quando há saldo folgado
  CarteiraService — reserva em custódia (RN06)
    OK   tira do saldo disponível e põe em custódia, sem sumir com o valor
    OK   registra um lançamento de custódia no livro-razão
    OK   recusa reserva que violaria o limite de dívida
  CarteiraService — devolução no cancelamento (RN07)
    OK   devolve integralmente o que estava retido
  CarteiraService — transferência na conclusão
    OK   debita a custódia da origem e credita o destino
    OK   grava sempre duas linhas no livro-razão, nunca uma só
    OK   trava as linhas sempre na ordem crescente de id, para evitar impasse
    OK   recusa transferência sem créditos reservados
    OK   recusa transferência para si mesmo
  CarteiraService — bônus de boas-vindas (RN02)
    OK   credita 3 Elos na aprovação do cadastro

ARQUIVO: unitarios/troca.test.js
  Máquina de estados da troca
    OK   pendente pode ir para aceita ou cancelada
    OK   pendente não pode ir direto para concluída
    OK   concluída é estado final
  UC05 — solicitar troca
    OK   cria a solicitação com status pendente e custo calculado pela duração
    OK   nenhum crédito é movimentado na solicitação
    OK   CT02 — recusa quando o saldo está no limite de dívida
    OK   CT05 — recusa durante período bloqueado pelo síndico
    OK   recusa habilidade desativada
    OK   recusa troca entre condomínios diferentes
    OK   recusa solicitar a própria habilidade
  UC06 — aceitar e recusar
    OK   aceitar reserva os créditos em custódia
    OK   só o ofertante pode aceitar
    OK   recusar não movimenta crédito nenhum
  UC07 — conclusão com dupla confirmação (RN04)
    OK   CT03a — uma confirmação sozinha não transfere nada
    OK   CT03b — a segunda confirmação transfere e conclui
    OK   a ordem das confirmações não altera o resultado
    OK   confirmar duas vezes é recusado
    OK   quem não participa da troca não pode confirmar
    OK   não é possível confirmar troca já concluída
    OK   CT04 — falha durante a transferência reverte tudo
  Cancelamento
    OK   cancelar troca aceita devolve os créditos retidos
    OK   cancelamento exige justificativa
    OK   não é possível cancelar troca já concluída
  Invariante global da economia
    OK   o total de Elos do condomínio não muda com uma troca concluída

ARQUIVO: integracao/api.test.js
  Saúde e autenticação
    OK   GET /api/saude responde sem token
    OK   rota protegida sem token devolve 401
    OK   token inválido devolve 401
    OK   login com senha errada não revela se o e-mail existe
    OK   campos obrigatórios faltando devolvem 422 com a lista
  Cadastro e aprovação pelo síndico (UC01, UC11)
    OK   cadastro nasce pendente e não consegue entrar
```

```
    OK   senha fraca é recusada antes de criar a conta
    OK   aprovação do síndico credita o bônus de boas-vindas (RN02)
    OK   morador comum não acessa a área do síndico
  Habilidades e busca (UC02, UC04)
    OK   publica habilidade com custo derivado da duração (RN01)
    OK   descrição curta demais é recusada com mensagem útil
    OK   a busca só devolve habilidades do próprio condomínio
  Ciclo completo de uma troca (UC05 a UC10)
    OK   do pedido à conclusão, com os saldos conferindo no fim
    OK   quem não participa da troca não lê o chat nem confirma
    OK   avaliação antes da conclusão é recusada (RN09)
    OK   cancelar depois do aceite devolve os Elos retidos (RN07)
    OK   pedir habilidade de outro condomínio é recusado
  Painel do síndico (UC13, UC14)
    OK   mostra indicadores e as lacunas de habilidade do prédio
    OK   suspensão das trocas bloqueia novos pedidos (RN10)
  Rota inexistente
    OK   fora de /api devolve 404 em JSON, não página de erro
    OK   dentro de /api, sem token, responde 401 antes de dizer se a rota existe
    OK   dentro de /api, autenticado, devolve 404

ARQUIVO: integracao/moderacao.test.js
  Sessão
    OK   GET /api/auth/eu devolve o morador do token, sem a senha
  Catálogo do morador
    OK   lista as próprias habilidades e consegue despublicar
    OK   ninguém despublica habilidade de outro morador
    OK   as categorias saem prontas para o filtro da busca
    OK   necessidade registrada aparece na lista do condomínio (UC03)
  Denúncia e apuração (UC16, UC12)
    OK   relato curto demais é recusado
    OK   não é possível denunciar a si mesmo
    OK   denúncia aberta chega ao síndico e não suspende ninguém sozinha
    OK   situação inválida na decisão é recusada
    OK   suspensão impede o morador de entrar e não vale para o próprio síndico
    OK   recusa de cadastro não credita bônus
    OK   cadastro já decidido não é decidido de novo
```

5.5.6 Resultado dos casos de teste funcionais

Caso	Descrição	Resultado obtido	Situação
CT01	Cadastro com código de condomínio inválido	O sistema recusa o cadastro e mantém os dados preenchidos	Aprovado
CT02	Solicitação bloqueada por limite de dívida	Com saldo em -3, a solicitação é recusada com a mensagem "Saldo insuficiente. Ofereça uma habilidade para voltar a pedir." e nenhuma troca é criada	Aprovado
CT03	Transferência só ocorre após a dupla confirmação	Após a primeira confirmação os saldos permanecem em 3 e 2 Elos e o extrato segue vazio. Após a segunda, os saldos passam a 3 e 4, a troca fica concluída e duas linhas são gravadas no livro-razão	Aprovado
CT04	Reversão da transação em caso de falha	Com falha forçada na transferência, os saldos permanecem em 3 e 2, a confirmação do ofertante não é gravada, o status continua aceita e o extrato segue vazio	Aprovado
CT05	Bloqueio de período preserva trocas já aceitas	Nova solicitação recusada com a mensagem de suspensão; a troca já aceita conclui normalmente	Aprovado

Além dos cinco casos documentados, a suíte verifica um invariante global da economia: o total de Elos em circulação no condomínio, somando saldo disponível e custódia de todos os moradores, permanece inalterado depois de uma troca concluída. Esse teste é a proteção mais forte contra criação ou destruição acidental de créditos, porque falha mesmo que um defeito futuro introduza um caminho de transferência que ninguém previu.

5.5.7 Defeitos encontrados e corrigidos

#	Defeito	Severidade	Correção	Situação
D01	A confirmação era gravada antes da verificação de duplicidade, permitindo que a mesma parte confirmasse duas vezes e disparasse a transferência sozinha	Crítica	A verificação do campo de confirmação passou a anteceder a gravação	Corrigido
D02	A transferência travava as linhas na ordem dos participantes, o que podia gerar impasse entre duas trocas simultâneas com as mesmas pessoas em sentidos opostos	Alta	As linhas passaram a ser travadas sempre em ordem crescente de identificador, independentemente do sentido da transferência	Corrigido
D03	O custo era lido da habilidade no momento da conclusão, permitindo que o ofertante alterasse o preço depois do combinado	Média	O custo passou a ser congelado na criação da troca	Corrigido

O defeito D02 foi encontrado por análise durante a escrita do teste de ordem de travamento, e não em execução. Ele é do tipo que só se manifestaria em produção sob concorrência, e dificilmente apareceria em teste manual.

5.5.8 Resultados da validação com usuários

A PREENCHER — Preencher após as sessões: taxa de conclusão por tarefa, tempo médio, número de pedidos de ajuda, pontuação SUS e observações qualitativas, com atenção especial ao desempenho dos participantes de baixa afinidade digital nas tarefas T1 e T5.

6. Conclusão

6.1 Objetivos alcançados

O projeto partiu de uma observação simples: condomínios concentram competências que ninguém enxerga, e por isso recorrem ao mercado externo para resolver o que o vizinho resolveria. O objetivo geral era desenvolver um aplicativo que tornasse essas competências visíveis e trocáveis por meio de créditos sem valor monetário, com painel administrativo para o síndico.

Objetivo	Situação	Evidência
Histórias de usuário e casos de uso	Concluído	27 histórias com critérios de aceitação, 16 casos de uso e matriz de rastreabilidade cobrindo os 26 requisitos
Interface e experiência	Concluído	Protótipo navegável com 16 telas, design system definido e decisões de acessibilidade rastreadas às personas
Base de dados	Concluído	Modelo entidade-relacionamento, dicionário de dados, oito regras de integridade e estratégia transacional
Implementação do núcleo	Preencher	Funcionalidades da Onda 1 em Flutter com API em Node.js e PostgreSQL
Testes e validação	Preencher	Casos de teste documentados, cobertura automatizada e sessões com moradores
Pitch e documentação	Concluído	Este documento e o roteiro da seção 6.4

6.2 Limitações

O registro honesto das limitações é parte do resultado, e três delas merecem destaque.

A primeira é de liquidez, e foi identificada ainda na fase de concepção. Em um prédio de 96 unidades com 40% de adesão há cerca de 38 usuários ativos. Como cada morador precisa de uma habilidade específica poucas vezes por ano, o volume realista fica entre 5 e 8 trocas mensais, bem abaixo da meta inicial de 30. Um aplicativo aberto uma vez a cada dois meses não retém usuários. A resposta do projeto foi incorporar oficinas abertas ao escopo, em que uma oferta atende vários moradores de uma vez, posicionadas na Onda 2 do sequenciador.

A segunda é de modelo de negócio. O produto proíbe dinheiro por definição, então a receita não pode vir do morador. Vender assinatura ao condomínio é possível mas caro, porque cada contrato depende de aprovação em assembleia. A alternativa avaliada foi deixar de ser um produto autônomo e integrar-se como módulo de uma plataforma de gestão condominial já existente, o que troca 167 negociações por uma.

A terceira é de escopo técnico. Ficaram fora da entrega a retenção de créditos em custódia, o chat em tempo real, as notificações push e a Árvore de Talentos em sua forma visual completa. Nos quatro casos a decisão foi deliberada e está registrada no sequenciador: entregar primeiro uma versão simples que resolve o essencial, e adiar a complexidade até que o uso real justifique o investimento.

6.3 Trabalhos futuros

Evolução	Justificativa
Oficinas abertas com inscrição	Ataca diretamente a restrição de liquidez, transformando uma oferta em várias trocas simultâneas
Custódia de créditos	Impede que um morador gaste créditos já comprometidos em troca pendente

Evolução	Justificativa
Árvore de Talentos visual	Transforma a lista de lacunas em visualização hierárquica, que é o diferencial competitivo do produto
Chat em tempo real e notificações push	Reduz o tempo de resposta entre pedido e aceite, que é o gargalo da experiência
Integração com plataforma de gestão condominial	Viabiliza a distribuição sem depender de venda condomínio a condomínio
Operação multi-condomínio	O modelo de dados já está preparado; falta a camada de administração entre condomínios

6.4 Roteiro do pitch

Apresentação de oito minutos, estruturada para que o problema seja sentido antes de a solução aparecer.

Tempo	Momento	Conteúdo
0:00 – 1:00	O gancho	Seu prédio tem um professor de violão no quarto andar e alguém que conserta computadores no nono. Ninguém sabe disso. Apresentar o número: mais de 520 mil condomínios no país vivendo essa situação
1:00 – 2:30	O problema	As duas personas lado a lado: Camila, que paga por um serviço que o vizinho faria, e Seu Antônio, que ajuda de graça e nunca recebe de volta. Mostrar as jornadas com as dores críticas destacadas
2:30 – 4:00	A solução	Demonstração ao vivo do fluxo completo: buscar, solicitar, combinar pelo chat, confirmar e ver o saldo mudar. Sem slides nesta parte
4:00 – 5:00	O diferencial	O painel do síndico e a visão de lacunas. É o que separa o Trocaí de um grupo de WhatsApp organizado
5:00 – 6:00	As decisões difíceis	Por que não há dinheiro no sistema, por que a Árvore de Talentos ficou para depois e por que as oficinas entraram no escopo
6:00 – 7:00	Validação	Resultados dos testes com moradores reais, incluindo o que não funcionou
7:00 – 8:00	Próximos passos	Roadmap das ondas 2 e 3 e a hipótese de distribuição como módulo integrado

Recomendação de condução: a demonstração ao vivo deve ocupar o centro da apresentação e ser ensaiada com dados já carregados, para que nenhum tempo seja gasto preenchendo formulários diante da banca.

7. Referências

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais. Brasília: Diário Oficial da União, 2018.

CAHN, Edgar S. No more throw-away people: the co-production imperative. 2. ed. Washington: Essential Books, 2004.

CAROLI, Paulo. Lean Inception: como alinhar pessoas e construir o produto certo. São Paulo: Caroli.org, 2018.

FLUTTER. Flutter documentation. Disponível em: https://docs.flutter.dev. Acesso em: 3 nov. 2026.

GOOGLE. Material Design 3: accessibility. Disponível em: https://m3.material.io/foundations/accessible-design. Acesso em: 3 nov. 2026.

INSTITUTO NACIONAL DE CONDOMÍNIOS E APOIO AOS CONDÔMINOS. Dados do setor condominial brasileiro. 2024.

JEST. Jest documentation: testing framework. Disponível em: https://jestjs.io/docs/getting-started. Acesso em: 3 nov. 2026.

NIELSEN, Jakob. 10 usability heuristics for user interface design. Nielsen Norman Group, 1994. Disponível em: https://www.nngroup.com/articles/ten-usability-heuristics. Acesso em: 3 nov. 2026.

ORGANIZAÇÃO DAS NAÇÕES UNIDAS. Transformando nosso mundo: a Agenda 2030 para o Desenvolvimento Sustentável. Nova York: ONU, 2015.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL documentation: transactions. Disponível em: https://www.postgresql.org/docs. Acesso em: 3 nov. 2026.

WORLD WIDE WEB CONSORTIUM. Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation, 2018. Disponível em: https://www.w3.org/TR/WCAG21. Acesso em: 3 nov. 2026.